

Comparing Open-Source and Cloud-Native AI/ML Security Tools as a Pre-Pipeline Gate in DevSecOps CI/CD Pipelines: A Study of SME Suitability

Thesis presented for the award of
Master of Science degree

Rakesh Uday Kumar
Student ID: A00047386

Supervisor: Dr. Shivani Jaswal

School of Enterprise Computing and Digital Transformation
Technological University Dublin, Tallaght Campus

31st August 2026

DECLARATION

I hereby certify that the material, which I now submit for assessment on the programmes of study leading to the award of Master of Science, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution.

Rakesh Uday Kumar

31st August 2026

Signature of Candidate

Date

ABSTRACT

Small and medium enterprises (SMEs) don't have the same security team resources as large enterprises, so the value of these AI/ML-powered security tools compared to traditional signature-based ones is not yet proven. This study aims to examine this question empirically by creating, deploying and directly comparing two multi-layer DevSecOps security toolchains: an open-source one (Semgrep, Trivy, Trufflehog, Falco, Open Policy Agent) and an Azure cloud-based one (CodeQL, Dependabot, Microsoft Defender for Cloud, Microsoft Sentinel, Azure Policy). Both toolchains were assessed taking into account six criteria: detection capability, false positive rate, setup complexity, pipeline overhead, cost and SME suitability, using a Design Science Research methodology.

The results indicate that CodeQL achieved significantly higher structural finding coverage than its open-source counterpart, depending on the configuration used. This benefit comes with a tangible cost, measured in time: Azure CI workflow, which is architecturally driven by CodeQL, took 9.7 times the baseline execution time, whereas the open-source workflow took 3.5 times; there were six setup friction points to two in Azure, and materially more ongoing interpretive complexity, as seen in the repeated cross-portal reporting inconsistencies in Azure Defender for Cloud, Sentinel, and Azure Policy. Most importantly, Azure's most unique and touted feature, AI/ML-augmented behavioural detection via Sentinel, was not used under adversarial conditions, as there is no attack that produces the kind of telemetry that could be put to the test. This study finds that there was no demonstrated AI/ML specific detection advantage from the measured costs of the Azure stack; it is identified as a limitation and direction.

Keywords : DevSecOps, AI/ML-enabled security, cloud-native security, CI/CD pipelines, security toolchains, SME security, behavioural anomaly detection.

ACKNOWLEDGEMENTS

First and foremost, I would like to thank my parents, Yashoda and Uday Kumar, and my sister, Navya, for their unwavering support and encouragement throughout this Master's degree programme. Without your understanding, patience, and constant belief in me, I would not have been able to achieve what I have today.

To my former colleagues at Accenture India Private Limited (Société Générale project) and Nice Interactive Solution Limited, Pune, whose dedication to continuous learning instilled in me the drive to return to study, and who gave me the space and confidence to take on that challenge.

I would also like to thank all the lecturers and staff of Technological University Dublin who taught me throughout this programme, for the knowledge and perspective they shared along the way.

Finally, I would also like to take this opportunity to express my gratitude and thanks to my supervisor, Dr. Shivani Jaswal, for her continued encouragement, motivation, and guidance during my research work.

Contents

DECLARATION	2
ABSTRACT	3
ACKNOWLEDGEMENTS	4
LIST OF TABLES AND FIGURES	7
ABBREVIATIONS	8
1. Introduction	9
1.1 Background	9
1.2 Research Question and Hypothesis	10
1.3 Aims and Objectives	11
1.4 Thesis Structure	11
1.5 Chapter Summary	11
2. Literature Review	12
2.1 Introduction	12
2.2 Foundational and Methodological Literature	12
2.3 SME and CI/CD Adoption Literature	14
2.4 Current DevSecOps and AI/ML Tooling Literature	15
2.5 Research Gap and Study Positioning	17
2.6 Chapter Summary	19
3. Research Methodology	20
3.1 Research Design	20
3.2 System Architecture	21
3.3 Open-Source Stack Design	23
3.4 Azure Cloud-Native Stack Design	23
3.5 Evaluation Metrics	23
3.6 Ethical and Data-Protection Considerations	25
3.7 Evolution from the Original Research Proposal	26
3.8 Chapter Summary	27
4. Implementation	28
4.1 Environment Setup	28
4.2 Open-Source Stack Implementation	28
4.2.1 Semgrep	29
4.2.2 Trivy	30
4.2.3 Trufflehog	31
4.2.4 Falco	32
4.2.5 Open Policy Agent (OPA)	33
4.3 Azure Cloud-Native Stack Implementation	34
4.3.1 CodeQL (GitHub Advanced Security)	34
4.3.2 Dependabot	35
4.3.3 Microsoft Defender for Cloud	37
4.3.4 Microsoft Sentinel	39
4.3.5 Azure Policy	42

4.4 Pipeline Integration and Verification.....	43
4.5 Chapter Summary	45
5. Comparative Analysis.....	46
5.1 Detection Capability	46
5.2 False Positive Rate.....	47
5.3 Setup Complexity	49
.....	49
5.4 Pipeline Overhead.....	49
5.5 Cost.....	50
5.6 SME Suitability	51
5.7 Synthesis Against the Study's Hypotheses.....	51
5.8 Chapter Summary	53
6. Discussion.....	54
6.1 Summary of Findings and Answer to the Research Question.....	54
6.2 Strengths and Limitations	55
6.3 Relation to the Literature	57
6.4 Practical Implications for SMEs	57
6.5 Chapter Summary	58
7. Conclusions	59
7.1 Contribution.....	59
7.2 Future Work.....	60
7.3 Closing Statement.....	61
REFERENCES	62
APPENDICES	65
Appendix A: OPA Rego Policies.....	65
Appendix B: Baseline CI/CD Pipeline Workflow	66

LIST OF TABLES AND FIGURES

List of Tables

Table 2.1: Comparison of Reviewed Literature.....	18
Table 3.1: Key design decisions and trade-offs.....	25
Table 3.2: Evolution from the original CA2 research proposal.....	26
Table 5.1: Detection capability by layer, open-source vs Azure stack.....	46
Table 5.1a: Comparability of evaluated controls by layer.....	47
Table 5.2: OPA policy evidence base — three risk classes, six ground-truth cases.....	47
Table 5.3: Documented friction points by stack.....	49
Table 5.4: Pipeline execution time, three interleaved trials.....	49
Table 5.5: Cost comparison, open-source vs Azure stack.....	50
Table 5.6: SME suitability synthesis across all preceding metrics.....	51
Table 5.7: Research question, hypothesis, and metric synthesis.....	52
Table 5.8: Summary of hypothesis outcomes.....	53
Table 6.1: Summary of comparative results by metric.....	54

List of Figures

Figure 2.1: Comparison of open-source and Azure cloud-native security stack components.....	17
Figure 3.1: Research process overview, from research design through to discussion and conclusions.....	20
Figure 3.2: Research methodology workflow, dual-stack architecture, and evaluation framework.....	21
Figure 3.3: Controlled experimental design and evaluation flow.....	15
Figure 4.1: Semgrep scan results against WebGoat - 20 findings across 974 files.....	23
Figure 4.2: Trivy scan results against the built WebGoat Docker image — 62 vulnerabilities.....	24
Figure 4.3: Trufflehog controlled test result - 2 of 4 deliberately planted secrets detected.....	25
Figure 4.4: Falco detection of an interactive shell spawned inside the running WebGoat container ...	26
Figure 4.5: OPA policy evaluation correctly flagging the untrusted-context shell-injection pattern (true positive).....	27
Figure 4.6: OPA policy evaluation against a clean control workflow, returning no finding (true negative).....	27
Figure 4.7: CodeQL scan results against WebGoat -71 findings (3 Critical, 52 High, 16 Medium)....	28
Figure 4.8: Dependabot alerts showing zero findings despite known vulnerable dependencies.....	30
Figure 4.9: WebGoat's upstream repository confirming the same vulnerable XStream dependency ...	31
Figure 4.10: Microsoft Defender for Cloud DevOps Security overview — 75 findings.....	32
Figure 4.11: Microsoft Defender portal showing 1 connector, 1 connected.....	34
Figure 4.12: Microsoft Sentinel's own Azure-native blade, same workspace, showing 7 connectors..	35
Figure 4.13: Microsoft Sentinel Incidents view showing zero incidents.....	36
Figure 4.14: Azure Policy compliance overview showing multiple, non-reconciled compliance metrics.....	37

ABBREVIATIONS

AI Artificial Intelligence

CI/CD Continuous Integration / Continuous Delivery

CVE Common Vulnerabilities and Exposures

DevSecOps Development, Security and Operations

DSR Design Science Research

GDPR General Data Protection Regulation

GHAS GitHub Advanced Security

IaC Infrastructure as Code

ML Machine Learning

NIST National Institute of Standards and Technology

OPA Open Policy Agent

SAST Static Application Security Testing

SIEM Security Information and Event Management

SME Small and Medium Enterprise

SSDF Secure Software Development Framework

STRIDE Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege

TN True Negative

TP True Positive

VM Virtual Machine

1. Introduction

1.1 Background

The 2020 SolarWinds SUNBURST attack proved how ineffective rule-based security solutions are when it comes to discovering malicious code that is added directly to a CI/CD pipeline. Most importantly, SUNBURST was built to bypass signature and behavioural detection and anomaly detection techniques, as well as signature-based tools, by utilizing a ‘dormancy period’ that exploits the fact that most tools and signatures are developed during an active threat window, and hence fail to identify dormant threats (FireEye, 2020).

The question explored in this study but left unassumed in the literature is whether AI/ML powered cloud-native tooling provides any meaningful protection for small and medium enterprises (SMEs) against this class of risk, or whether the increased complexity and complexity of implementation outweighs any advantage the tool will provide in a real environment where the supply chain is compromised. The existing AI/ML based security systems are generally enterprise-oriented and are not suitable for small and medium enterprises (SMEs) because of their cost and infrastructure requirements, as is the case in Mahimalur et al. (2025), which assumes enterprise scale infrastructure (Kubernetes, enterprise IAM, distributed streaming platforms).

The DevSecOps movement has aimed to fill this type of gap, by integrating security into the development process from its beginning stages through the end of the process as a continuous activity (Myrbakken and Colomo-Palacios, 2017). A pre-pipeline security gate – a security check completed prior to the passage of code or infrastructure through the pipeline – is one of the first and most inexpensive places in the pipeline to detect a compromise in the supply chain. However, in practice, the tooling options to connect to such a gate run two basically distinct routes: open-source, self-managed toolchains with the need to setup and maintain the tools in-house; and cloud-native, vendor-managed options that are expected to ease up the operational burden in the short-term, but at the expense of subscription fees and platform lock-in. If a large enterprise has its own security engineering team, this may be a secondary consideration for them when choosing between two different solutions based on detection ability. However, for an SME with a small budget and no security team, the selection of the toolchain becomes a strategic decision in itself, one that can impact whether a pre-pipeline security gate is ever selected and maintained or never even considered because it is too expensive and/or complex to maintain.

Cloud vendors are selling these security tools based on embedded artificial intelligence/machine learning (AI/ML) features that include behavioural anomaly detection, machine-learning-driven vulnerability prioritisation, and automated correlation of signals that would otherwise need to be manually correlated by a dedicated security analyst. These are often offered as valuable enhancements to typical signature and rule-based static analysis. The empirical evidence that such AI/ML-augmented tooling provides a proportionate benefit to SME-scale deployments rather than enterprise deployments with the right amount of telemetry and staff can be operationalising it, however, is sparse and as Chapter 2 sets out, it is not always the case. This study does not accept this as a premise, rather it is a hypothesis to be tested empirically, and it does this by creating and testing both toolchains against a realistic and vulnerable test application under similar conditions.

The cloud-native toolchain described in this research study was originally designed around AWS-native security services within AWS (as specified in the CA2 research proposal), but was modified to the Azure-based toolchain evaluated in this study with the supervisor's approval due to limitations on the duration of AWS's free-trial access compared with the evaluation period for this study.

1.2 Research Question and Hypothesis

Is it possible to integrate a lightweight AI/ML pre-pipeline security gate into a DevSecOps CI/CD pipeline to measurably enhance security results for SMEs without excessive complexity or overhead?

The review of the literature in Chapter 2 produced two testable hypotheses, which are outlined here:

H1: The open-source toolchain will present the lower setup complexity, lower pipeline overhead, and lower cost than the Azure cloud-native toolchain, as reported by the adoption friction of using tools reported by Rajapakse et al. (2022) as the single most frequently cited barrier to DevSecOps adoption, and the lack of validated SME scaling guidance reported by Ccallo-Luque and Quispe-Quispe (2024).

H2: Both stacks use the same structural-analysis principles in a combination of core static analysis tools (Semgrep and CodeQL): If one stack has an advantage in the structural-analysis parts of their respective tooling, it is only likely to be apparent in the Azure stack's AI/ML-augmented parts of the tooling, that is, in the behaviour and anomaly-detection layers of Defender for Cloud and Microsoft Sentinel; and as Wienczkowski (2026) has identified, the structural analysis and adaptive analysis approaches are not

necessarily incompatible, and this advantage should not be taken for granted until tested directly.

1.3 Aims and Objectives

This project constructs and experiments two multi-layer DevSecOps security toolchains, one based on open-source tools and the other based on cloud-native services; and examines which one is really appropriate for SMEs. The toolchains are both tested against a common test artefact set, tested on the same evaluation framework, and integrated in operational CI/CD pipelines, while direct quantitative comparisons are made only for technically similar control layers, and are compared against six metrics: Detection capability, False positive rate, Setup complexity, Pipeline overhead, Cost, and SME suitability.

1.4 Thesis Structure

Chapter 2 summarizes the available literature on DevSecOps adoption, CI/CD security tooling and AI/ML-enabled security testing, which provides the context for the empirical gap addressed in this study. Chapter 3 presents the Design Science Research methodology used, the system architecture developed for the design of the two toolchains, and the six evaluation metrics employed. The implementation of both stacks is detailed in Chapter 4. Chapter 5 provides a comparison of all the metrics. These findings are discussed with the research question, in comparison with the literature and with the strengths and weaknesses of the study in chapter 6. The study contribution and future directions are discussed in the last chapter.

1.5 Chapter Summary

A key motivation for this study is the SolarWinds SUNBURST incident, which showed that a well-crafted supply-chain attack can easily get past a signature-based as well as a behavioural-based detection system, making the question of the increased value of AI/ML-driven cloud-native security tooling for SMEs versus the increased cost and complexity of implementing it very real. This paper thus questions that empirically by constructing and comparing two security stacks, one based on open source and the other based on Azure cloud, on the same shared test artefact, in six comparative metrics. The rest of this thesis documents the literature that was reviewed, the method used, and the implementation, results, and conclusions (later chapters).

2. Literature Review

2.1 Introduction

This chapter summarizes the literature that underpins this study in three categories: foundational and methodological studies that have investigated DevSecOps and static analysis tooling from an academic perspective; literature devoted to the challenges of SME adoption of CI/CD; and recent works on DevSecOps tooling and how to augment security testing with AI/ML. The chapter culminates with situating the unique empirical contribution of this work in comparison with each of the sources examined.

2.2 Foundational and Methodological Literature

One of the earliest academic works on DevSecOps is by Myrbakken and Colomo-Palacios (2017) where they describe DevSecOps as an extension of DevOps practice, introducing security activities at every stage of the development lifecycle, and not as a gate at the end. The practitioners' and grey literature heavily influences their multivocal literature review, which they themselves admit as a limitation, so that the definition and characterisation of DevSecOps is used in subsequent academic literature like that of Rajapakse et al. (2022) as a reference point. Myrbakken and Colomo-Palacios demonstrate that DevSecOps, as a field, has always been about tooling and automation as its means of adoption, and for this study, they choose to focus on the direct comparison of tools instead of relying on what people have perceived as DevSecOps.

Lipp, Banescu and Pretschner (2022) offer a methodologically sound precedent for comparing static analysis tools to real vulnerabilities instead of synthetic vulnerabilities, and for comparing different C/C++ analyzers to a collection of real-world CVEs instead of a benchmark suite of artificially inserted flaws. It is also worth noting that their main methodological thesis — the use of synthetic data sets which lack the depth and complexity of security bugs in real-world applications — directly anticipates and enables the choice of OWASP WebGoat, a realistic and not purpose-built-for-labelling test artefact, over a fully synthetic test artefact like OWASP Benchmark (to be discussed further in Section 6.2 as a limitation of the study). The study does focus on using a single language (C/C++) and a single tool category (SAST), but it does not cover additional tool categories, such as dependency scanning, secret detection, runtime monitoring, or policy enforcement, and it does not compare open-source and cloud-native toolchains as whole stacks, as this study does.

Charoenwet et al. (2024) provide a more directly comparable empirical study of five static application security testing tools, including one of those evaluated in this study, CodeQL, which is also part of the Azure stack. Most importantly, they conclude that SAST tools are significantly less successful at finding vulnerabilities in real vulnerability-contributing commits compared to the synthetic or fully vulnerable test programs typically employed in academic studies, and explicitly call for future comparative studies to take this difference between synthetic and real-world test conditions into account. This study is neither an end-to-end use of OWASP WebGoat, the purpose-built, ground-truth-labelled benchmark that would lend itself to the former extreme, nor an end-to-end use of OWASP Benchmark, the much simpler and less realistic benchmark that would lend itself to the latter extreme, and this distinction is restated as a limitation in Chapter 6.

The U.S. National Institute of Standards and Technology's Chandramouli, Kautz and Torres-Arias (2024) offer authoritative strategic guidance for using software supply chain security in DevSecOps CI/CD pipelines in NIST Special Publication 800-204D. Their framework also identifies the software supply chain as the complete sequence of steps starting with source, building and deploying it and suggests that security assurance needs to be intrinsic and pervasive along the entire pipeline, rather than just during the time of release. This document is not an empirical study—it is a strategic and normative document that prescribes what a secure CI/CD pipeline should consist of, but does not test, benchmark, and compare different tool implementations to each other. The supply-chain framing defined in NIST SP 800-204D is used in this study as a normative measure for evaluating the actual empirically measured behaviour of the two toolchains that are evaluated.

The following is a larger-scale empirical basis for the motivating threat case of this study other than the SolarWinds story in Section 1.1. There is also a large scale empirical basis for the motivating threat case for this study beyond what is introduced in Section 1.1 as the SolarWinds story. They analyse more than 320,000 CI/CD-configured GitHub repositories, showing that runtimes for CI/CD scripts are systematically vulnerable to code hiding, untimely updates, and even scripts from verified creators can be severely vulnerable, and that the ecosystem is heavily dependent on a small number of core scripts, creating single points of failure; they also validate five real-world attack scenarios directly against live CI/CD environments. Their research is not evaluative but measurement and taxonomy centric – they describe the size and nature of the CI/CD attack surface without determining if any of the detection tools in the market, open-source or cloud-native, would be able to detect the types of compromise they uncover. The

side-by-side comparison of ten security tools in this study directly compares the detection capability of the various tools against the kinds of pipeline-level threats that Pan et al. only describe in terms of prevalence and taxonomy.

2.3 SME and CI/CD Adoption Literature

Rajapakse et al. (2022) mapped 21 distinct adoption challenges and 31 proposed solutions in four thematic areas (People, Practices, Tools and Infrastructure) based upon 54 peer-reviewed studies on DevSecOps adoption. It is important to notice that in the literature they have reviewed, they find that the adoption problems which are the most frequently mentioned are problems with the tools, followed by organisational or cultural problems. Overall, this reinforces the central focus of this study's own framing of the Setup Complexity of the two toolchains as primary object of comparison as opposed to secondary, and the friction points that are identified in Chapter 5 stand as examples of the pattern of friction points that is identified in the literature in general by Rajapakse et al.

Direct large-scale quantitative evidence of the adoption gap that Rajapakse et al. describe qualitatively is provided by Ayala and Garcia (2023). They query more than 173,000 popular GitHub repositories to find that only 37% even have any form of CI/CD pipeline, and only 7% are building a documented security policy—showing that adoption of security-tooling is lagging significantly behind that of CI/CD, even for well-known open-source projects, and certainly for resource-constrained SMEs. Their study does not measure the actual uptake of what is being adopted; it does not measure the relative benefits of the range of tools that can be used for the small percentage of projects that do undertake security automation — what is measured directly in this study's comparison.

This is further restricted by Ccallo-Luque and Quispe-Quispe (2024) who systematically reviewed 13 studies and reduced it to Very Small Entities such as organisations with a similar size to the SMEs this study focuses on. They find that the adoption of CI/CD tools is never addressed by validated, adapted guidance in Very Small Entities, and express a clear desire for more work to come that will provide validation and adaptation of the guidance for Very Small Entities, not extrapolating from enterprise-scale practice. Their review, however, is still at the level of cataloging what is reported to have been done and what challenges have been noted, and they don't carry the task of actually doing and comparing the tooling. This is where this study fits in: between Ccallo-Luque and Quispe-Quispe, where they see the lack of guidance

is unsuitable for SMEs, this study offers a controlled and empirical comparison to help address exactly that guidance.

2.4 Current DevSecOps and AI/ML Tooling Literature

Cheenepalli et al. (2025) focus on the adoption of DevSecOps in SMEs and gather the hands-on insights and challenges that adopts practitioners are facing in small enterprises with limited resources. Although it clearly offers the required type of background to the framing of the research question used throughout this study, it is not a controlled comparison of the various pipelines; it is simply a survey of reported practitioner experience, which this study addresses directly by measuring detection capability, false positive rates, and pipeline overheads.

In a systematic review, Nair et al. (2025) show that Zero Trust Architecture is an effective solution to DevSecOps challenges as it provides identity and context-based access control, which is a more robust security approach than perimeter-based architectures, especially in distributed CI/CD systems. Although Zero Trust itself is not a focus of this study, Nair et al.'s broader thesis that the assumptions built into a tooling architecture and its related tools can not align with the demands of the team that would use them is equally relevant to this study: The tooling architecture in Azure is based on these assumptions, and the study found friction when it was applied by a small team without dedicated security staff.

In this combination, Mahimalur et al. (2025) explain that the combination of AI/ML and containerisation can be used to enhance coverage and responsiveness in cloud security automation. Unlike the claim of a general benefit in using AI/ML and automation together, which is assumed by Mahimalur et al., the work in this study explores this assumption empirically.

The most closely related recent research is by Wienczkowski (2026), which systematically reviews 55 peer-reviewed papers on adaptive and AI-assisted security testing, involving structural program analysis, feedback-driven testing, and mixed learning-based methods. The survey highlights a lack of synergy in the tools supporting rich/structural program analysis techniques (which are exemplified by the codeQL, which was assessed in this study) and adaptive/fbdriven detection methods reacting to runtime or behavioural signals. Wienczkowski does not view this division as a settled issue in the field, but rather a question that needs to be addressed. This study's own result, that Microsoft Defender for Cloud found that it significantly collapses the structural findings of CodeQL instead of adding anything of its own in terms of detection value during adversarial scenarios, and that the behavioural correlation engine of

Microsoft Sentinel was not actually used during the study under any adversarial conditions is an empirical example of the kind of fragmentation that Wienczkowski describes in his literature survey.

The most recent contribution reviewed in this study is Dhandapani (2025), which uses the STRIDE threat modelling framework for a representative CI/CD pipeline and correlates the threats mentioned by them to the actual mitigating tools. Motivated explicitly by the SolarWinds SUNBURST breach, this study's toolchain mapping mirrors the recommendations made by Dhandapani's - who are the creators of Falco, Trivy, and Dependabot, which form the four tools of this study's open-source stack, and Open Policy Agent for policy enforcement. However, the mapping is not done: There is no tool deployed, no pipeline run, no comparison is made between the open source and then the cloud-native option. This study directly extends Dhandapani's threat-to-tool mapping from design artefact to an executed, measured comparison to check if the coverage is provided by the tools recommended by the mapping.

2.5 Research Gap and Study Positioning

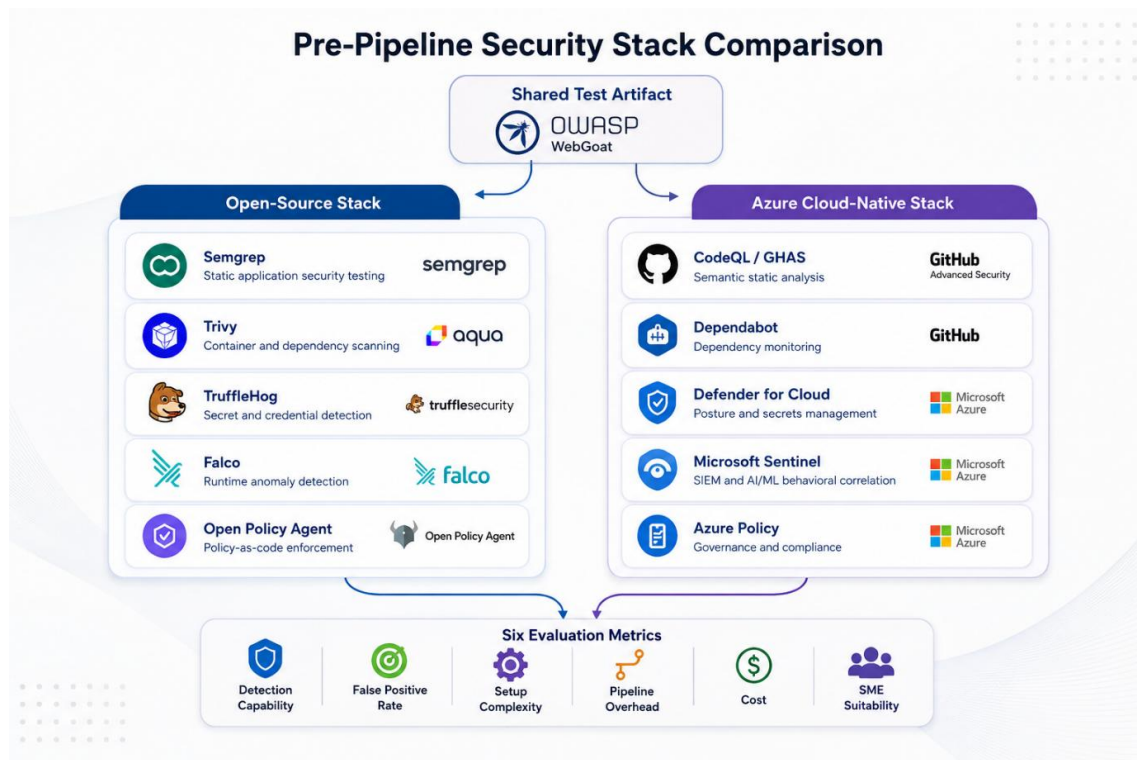


Figure 2.1: Comparison of open-source and Azure cloud-native security stack components

Compare open-source and Azure cloud-native security stack components. Find similarities and differences among open-source and cloud-native security stack components in Azure.

Ccallo-Luque and Quispe-Quispe (2024) confirm that there is no validated guidance on adopting CI/CD practices for Very Small Entities, but their study is limited to practice cataloguing and not in comparing the empirical tools, which this study directly sets out to do, by conducting both toolchains on the same test artefact in controlled side-by-side executions. This study's own false-positive analysis shows that false-positive rates are not uniformly distributed, but are systematically different by warning type as well, as found by Aloraini et al. (2019), but not by cloud-native or SME-cost. The work presented by Wienczkowski (2026) provides an overview of 55 studies and reveals a lack of integration between rich structural program analysis and adaptive, feedback-based detection; this structural vs. adaptive distinction was the focus of this study and the behavioural portion of the study went unexplored within the available telemetry conditions, as will be discussed in Sections 5.7 and 6.2.

All the literature reviewed does not present an open-source and a cloud-native AI/ML-augmented stack, deployed against the same test artefact, with the same complex setup

alongside the same pipeline overhead and SME cost. This is the empirical gap that the present study aims to fill.

Figure 2.1: **Comparison of Reviewed Literature**

Author/Year	Focus	Methodology	Key Finding	Gap Relative to This Study
Myrbakken & Colomo-Palacios (2017)	Foundational definition of DevSecOps	Multivocal literature review (practitioner + academic sources)	DevSecOps is fundamentally oriented around tooling and automation as its primary adoption mechanism	No empirical tool comparison; definitional/conceptual only
Aloraini et al. (2019)	SAST warning quality	Empirical study of static analysis warnings	False-positive rates vary systematically by warning type, not uniformly across a tool's output	Does not extend to cloud-native tooling or SME-cost dimensions
Rajapakse et al. (2022)	DevSecOps adoption challenges	Systematic review of 54 peer-reviewed studies	Tool-related challenges are the most frequently reported adoption barrier, ahead of culture/organisation	Identifies the barrier category but does not measure it empirically
Charoenwet et al. (2024)	SAST tool performance (incl. CodeQL)	Empirical evaluation on 1,060 vulnerable functions / 319 real vulnerabilities	SAST tools perform worse on real vulnerability-contributing commits than on synthetic benchmarks	Not SME-focused; does not compare open-source vs cloud-native stacks
Ccallo-Luque & Quispe-Quispe (2024)	CI/CD adoption in Very Small Entities	Systematic review of 13 studies	Very Small Entities lack validated, SME-scale CI/CD adoption guidance	Practice-cataloguing only; no tool execution or comparison
Cheenepalli et al. (2025)	DevSecOps in SMEs	Practitioner survey of challenges/best practices	Grounds SME-specific framing of adoption challenges	Survey-based; no controlled measurement of detection, FP rate, or overhead
Nair et al. (2025)	Zero Trust Architecture for DevSecOps	Systematic review	Architectural assumptions in tooling can mismatch operational team reality	Not focused on AI/ML tool comparison; conceptual argument only
Mahimalur et al. (2025)	AI/ML + containerisation for cloud security	Descriptive implementation case	Presents AI/ML and automation as straightforwardly improving detection and response	Benefit assumed rather than empirically tested; this study tests that assumption directly
Wienczkowski (2026)	AI-augmented security testing	Systematic survey of 55 studies	Persistent fragmentation between structural program analysis and adaptive, feedback-driven detection	Survey-level only; this study supplies a direct empirical instance of the fragmentation
Chandramouli, Kautz & Torres-Arias (2024)	Software supply chain security strategy	NIST Special Publication (normative framework)	Defines supply-chain security as a continuous pipeline property, not a release-time check	Strategic/normative only; does not test or compare specific tools

Dhandapani (2025)	STRIDE threat modelling of CI/CD pipelines	Threat-to-control mapping across pipeline stages	Recommends Falco, Trivy, Dependabot and OPA as mitigating tools for identified threats	Prescriptive only; no tool is deployed, run, or compared empirically
Lipp, Banescu & Pretschner (2022)	SAST effectiveness vs real CVEs	Empirical evaluation of C/C++ analyzers against real-world CVEs	Synthetic benchmarks do not reflect real vulnerability complexity	Single language, single tool category; no stack-level comparison
Ayala & Garcia (2023)	CI/CD workflow and security policy adoption	Large-scale query of 173,000+ GitHub repositories	Only 37% have CI/CD workflows enabled; just 7% have a security policy	Measures adoption prevalence only; does not assess detection outcomes
Pan et al. (2024)	CI/CD pipeline attack surfaces	Large-scale measurement of 320,000+ CI/CD-configured repositories	Script runtimes prone to code hiding; core-script reliance creates single points of failure	Measurement and taxonomy only; does not evaluate any detection tool

2.6 Chapter Summary

In this chapter, the literature has been reviewed from the basic definition of DevSecOps to the empirical evaluation of SAST tools to the challenges that need to be overcome by SMEs and Very Small Entities (VSEs) in adopting CI/CD to today's efforts on security testing with AI/ML assistance. In this literature, a consistent gap is revealed: adoption challenges and the capabilities of AI/ML-augmented tools and tooling have been discussed separately, but there is no existing study that compares an open-source and a cloud-native AI/ML-augmented security stack under controlled conditions that are compared side by side across the operational dimensions — cost, complexity of setup, and pipeline overhead — that will drive SME adoption. The methodology employed to meet this gap is presented in the next chapter.

3. Research Methodology

3.1 Research Design

The research methodology used in this study is the Design Science Research (DSR) methodology (Hevner et al., 2004) which is chosen because it is a research methodology that not only observes an existing phenomenon but also builds and evaluates a technical artefact. The evaluation phase of this DSR study also employs parts of the quasi-experimental comparison design approach where a baseline pipeline (no security tooling) is used as a control condition for measuring the operational overhead of each security toolchain.

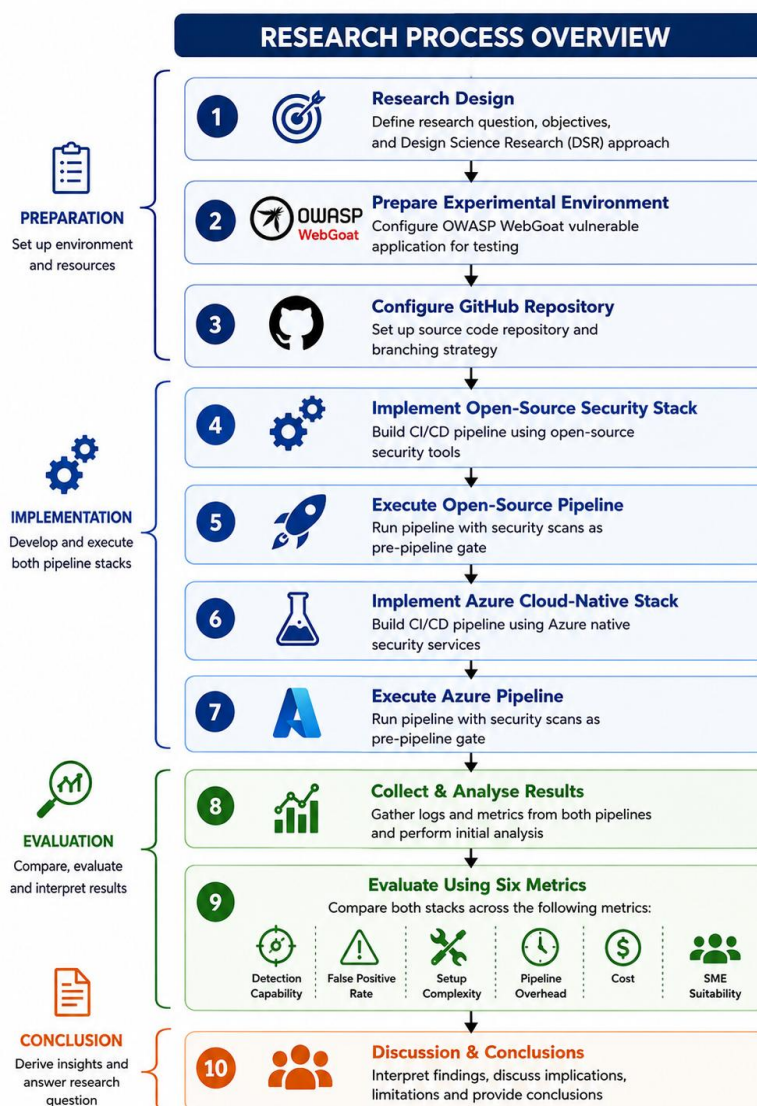


Figure 3.1: Research process overview, from research design through to discussion and conclusions

3.2 System Architecture

The OWASP WebGoat was chosen as the common test artefact for both toolchains, thus providing the same test conditions for both. WebGoat is designed to have realistic, intentional vulnerabilities which align with the OWASP Top 10 (OWASP, 2022), so it is available for evaluation without any need to inject vulnerabilities.

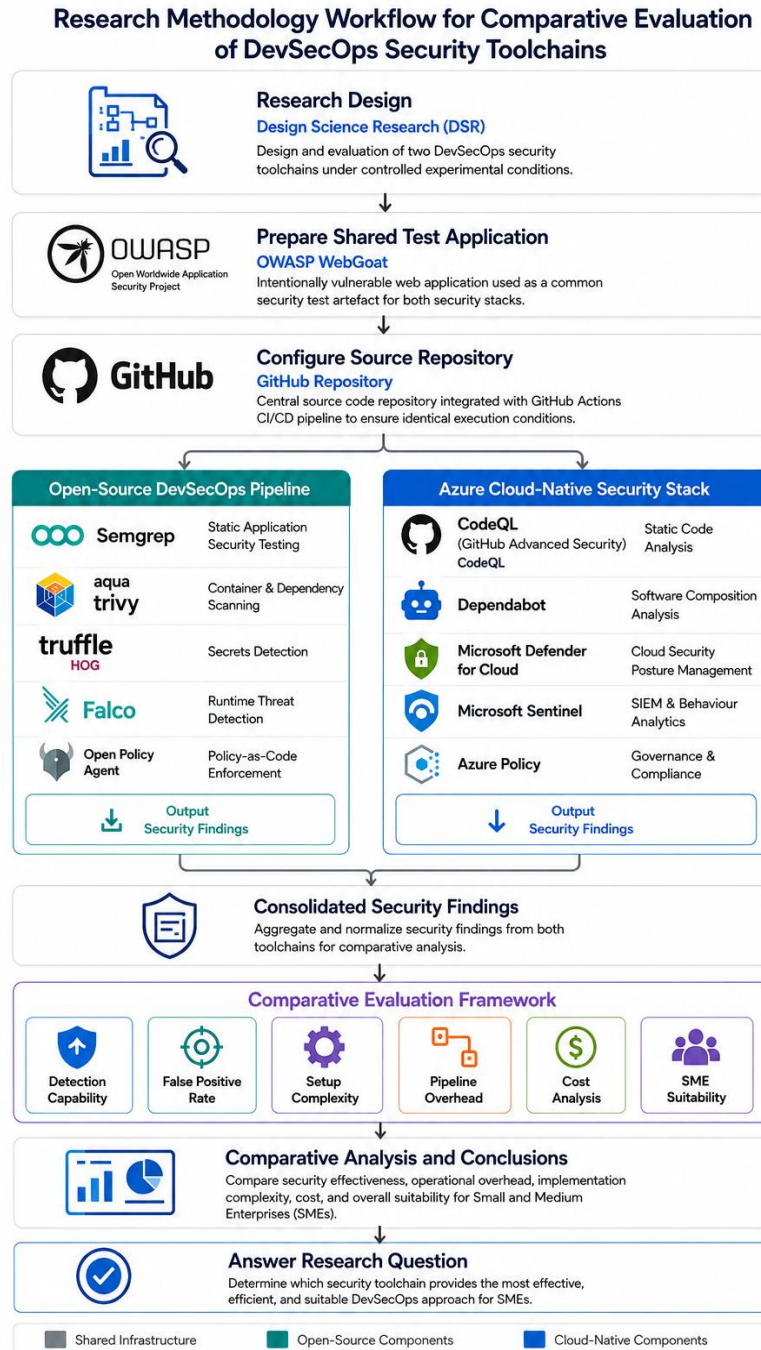


Figure 3.2: Research methodology workflow, dual-stack architecture, and evaluation framework

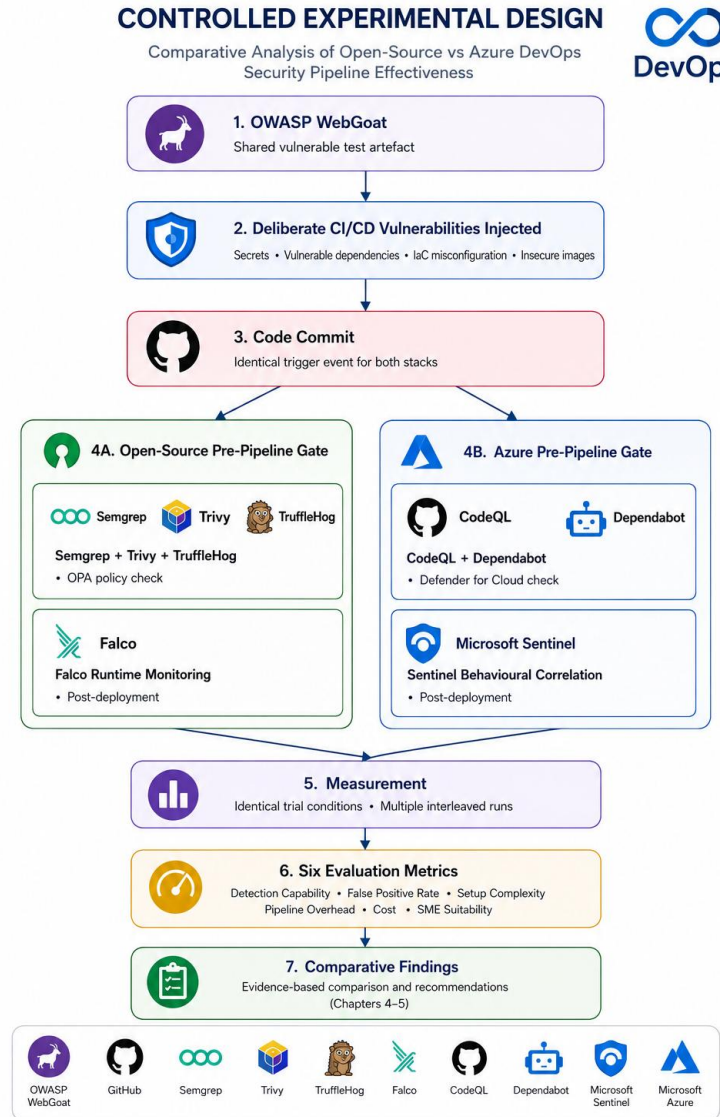


Figure 3.3: Controlled experimental design and evaluation flow

The experimental flow is detailed in Fig. 3.3, in which the same WebGoat artefact is run in both toolchains, with the same setup of trials, resulting in the 6-metric comparison presented in Chapters 4 and 5. All artefacts used in this study to support the implementation (configuration files, Rego policies, GitHub Actions workflows and complete screen evidence base throughout Chapter 4 and 5) are openly accessible at <https://github.com/Rakeshu99/devsecops-research>. This chapter explains in detail the contents of each stack, starting with the open source toolchain.

3.3 Open-Source Stack Design

The open-source stack consists of five tools that target different layers of the software supply chain: Semgrep (Semgrep, 2026) for static code analysis, Trivy (Aqua Security, 2026) for container and dependency scanning, Trufflehog (Truffle Security, 2026) for secret detection, Falco (The Falco Authors, 2026) for runtime anomaly detection, and Open Policy Agent (Open Policy Agent, 2026) for policy-as-code enforcement.

3.4 Azure Cloud-Native Stack Design

The Azure stack consists of five components: Azure CodeQL / GitHub Advanced Security (GitHub, 2026a) for semantic static analysis, Dependabot (GitHub, 2026b) for dependency monitoring, Azure Defender for Cloud (Microsoft, 2026a) for posture management, Azure Sentinel (Microsoft, 2026b) for SIEM and behavioural correlation, and Azure Policy (Microsoft, 2026c) for governance and compliance.

3.5 Evaluation Metrics

Before implementation, six metrics were established to understand both the technical performance of each toolchain and its practical usability for a SME with no dedicated security personnel.

Following Charoenwet et al. (2024) who cautioned finding counts cannot be directly compared between tools across different layers of the software supply chain, Detection Capability is measured as the number of findings generated, and summarized and detailed in qualitative terms as to what the findings represent when the tool is applied to the shared test artefact.

In this study false positive rate was evaluated by the deliberately constructed test cases, a pair of true positive and true negative examples for each policy or rule being assessed, instead of manual triage of the entire finding set, which was considered infeasible within the scope of this study. This is similar to Aloraini et al. (2019), who show that the false-positive rate for each type of finding differs from one another, and not consistently throughout a tool's output, thereby arguing for a type-by-type assessment of false positives in SAST tooling.

Setup Complexity is assessed qualitatively based on the actual setback issues that arise when implementing the tool, such as configuration problems, platform-specific behavior, and the amount of work required to diagnose the issues, and is not based on a checklist of expected issues as in the work of Rajapakse et al. (2022), who have identified adoption challenges related to the tools as heterogeneous and context dependent. The difficulty of diagnosis for each

friction point is nevertheless presented on a standard three level scale, which was subsequently applied after the actual problem was identified and presented in full below.

The difficulty of the diagnosis was defined as Low, if the problem could be solved directly from information presented within the tool's or platform's own interface; Medium, if the problem could be solved by referring to information in the documentation or the platform's API, or accessing more than one view within the platform; and High, if the problem could only be solved by inspecting raw logs or API output, or by diagnosing across multiple interacting systems, or when there was no indication within the tool's or platform's interface that a problem existed at all. This rubric was used for all the friction points that were identified in Chapter 4 and reported in Table 5.3, and not as an unspecified researcher judgement.

The wall-clock time that each toolchain consumes is measured in Pipeline Overhead, which includes a baseline CI/CD pipeline measured from a series of interlaced trials to eliminate runner and network noise and measure the overhead.

Cost is measured directly (Azure's actual data on the bills for the Azure stack's trial-based services) and structurally (the licensing model for each toolchain, open source versus commercial/trial) since the evaluation window of this study is completely in the free trial period of Azure.

SME Suitability is a synthesis judgement based on the following five metrics but is weighted towards the operational characteristics of cost certainty, result transparency and interpretive burden most relevant to an organisation without dedicated security expertise rather than just towards the raw detection depth.

The implementation documented in Chapter 4 is followed by results for all the six metrics in tabular form comparing the open-source and Azure cloud-native stacks against each of the metrics in Chapter 5.

The quantitative evidence base for this study is limited to descriptive, within-study comparison (with six ground-truth data points for False Positive Rate (Section 5.2) and three interleaved trials for Pipeline Overhead (Section 5.4), respectively), without statistical inference. Because of the small sample size, there are no confidence intervals or formal level of significance in the results reported in Chapter 5, but the conclusions drawn in this chapter shall be treated as being within the specific scope of this study and not as general claims, this scope of the study and its rationale are discussed further as a stated limitation in Section 6.2.

In this study we have drawn some major design decisions at each stage of our methodology, and these decisions are justified in their own context in the corresponding section, but brought together here as one reference point – see Table 3.1.

Decision	Alternative Considered	Rationale	Trade-off Accepted
OWASP WebGoat as shared test artefact	Purpose-built, ground-truth-labelled benchmark (e.g. OWASP Benchmark)	Realistic, <u>intentionally-vulnerable</u> code more closely reflects real vulnerability-contributing commits than synthetic test programs (Charoenwet et al., 2024)	Detection results cannot be checked against a fully known ground truth (Section 6.2)
Azure cloud-native stack	AWS-native stack (as originally proposed in CA2)	AWS free-trial access window was incompatible with this study's evaluation timeline; revised with supervisor approval (Section 1.1)	Loss of continuity with the <u>originally-proposed</u> AWS toolchain
Single test application, single language (Java/Spring Boot)	Multiple applications across different languages/frameworks	Enabled controlled, depth-focused comparison within a Master's-level timeframe	Generalisability to other technology stacks is not established (Section 6.2)
Descriptive rather than inferential statistics	Larger-N trials supporting confidence intervals/significance testing	Matched the timeframe realistically available; avoided overstating statistical rigor the sample sizes could not support	Findings are within-study evidence, not statistically generalisable claims (Section 3.5, 6.3)
OPA policy-as-code alongside Semgrep	Relying on Semgrep's built-in rules alone	Cross-validating the same finding via two independent detection mechanisms strengthens confidence it is genuine, not tool-specific (Section 4.2.5)	Additional policy-authoring effort required — itself a Setup Complexity finding
Six-metric comparison framework	A single composite "winner" score	Avoids collapsing genuinely different trade-offs (e.g. detection depth vs cost) into one obscuring number	Requires weighing six separate findings rather than one bottom-line answer

Table 3.1: Key design decisions and trade-offs

3.6 Ethical and Data-Protection Considerations

This study does not involve human participants, surveys, or interviews, nor is any personal real-life information, health information or other sensitive information collected, processed or stored at any time. The shared test artefact, OWASP WebGoat, is an intentionally vulnerable application made public by OWASP for security research and training purposes; vulnerabilities exposed, as well as additional weaknesses introduced for this study (hardcoded secrets, vulnerable dependencies, misconfigured workflow files), are synthetic and deliberately created for testing purposes and do not represent real credentials, real infrastructure secrets, or data belonging to any third party.

All the infrastructure utilized for this study is being used by the researcher on their own Azure for Students subscription and their own GitHub account. This study does not access, modify or risk any production systems, organisational infrastructure or third party services. The use of this research design does not trigger the data-protection requirements of the General Data Protection Regulation (Regulation (EU) 2016/679) that would apply to a study that would process real user data.

3.7 Evolution from the Original Research Proposal

The focus is now on how the original research proposal evolved. The emphasis is now on the evolution of the original research proposal.

There were a number of changes made to the original CA2 research proposal that resulted in the design of this study. To make deviations clear and traceable, each proposal commitment is summarised in Table 3.2 against the approach taken in this dissertation.

Proposal Commitment	Final Approach	Status / Rationale
AWS-native cloud stack (CodeGuru, Inspector, Macie, Security Hub, GuardDuty)	Azure-native cloud stack (CodeQL, Dependabot, Defender for Cloud, Azure Policy, Sentinel)	Changed with supervisor approval; AWS free-trial duration was incompatible with this study's evaluation timeline (Section 1.1)
OWASP WebGoat as shared test artefact	Retained	Unchanged
GitHub Actions CI/CD	Retained	Unchanged
Design Science Research methodology	Retained, with a baseline pipeline comparison added	Strengthened rather than reduced (Section 3.1)
Detection Capability, Setup Complexity, Pipeline Overhead, Cost	Retained as core metrics	Unchanged in substance; comparability boundaries made explicit (Table 5.1a)
Developer experience, measured by observing impact on developer workflow	Not retained as a distinct, separately measured metric	Absorbed into Setup Complexity and SME Suitability; disclosed explicitly below
False positive rate compared "for both stacks"	Narrowed to controlled, open-source (OPA) ground-truth validation only	No equivalent controlled ground-truth test was constructed for the Azure stack within this study's scope (Section 5.2)
Cost estimated at under €20 (AWS free tier)	€0 observed expenditure during the Azure free-trial evaluation (Azure Cost Management displayed US\$0.00)	Platform change altered the specific cost model; the underlying free/trial-based finding is consistent with the proposal's expectation
Two subsidiary research questions (RQ1: detection comparison; RQ2: developer workflow and overhead)	Two testable hypotheses (H1, H2)	RQ1 operationalised into H2; RQ2's overhead component operationalised into H1; RQ2's developer-workflow component not retained (see below)

Table 3.2: Evolution from the original CA2 research proposal

The initial suggestion was to make developer experience a separate assessment metric, to be assessed by evaluating developer workflow as they work through developer security testing. This criterion is not singled out and named as an outcome measure in the final study. Due to the nature of this dissertation (which would not have been possible in the scope of the work, or in the time limit set, to carry out a study of the developer-workflow compared to that of other participants), it was not possible to use a study of multiple participants. The friction points and diagnostic effort recorded in section 5.3 (Setup Complexity) and the interpretive burden recorded in section 5.6 (SME Suitability) cover a closely related dimension of this original criterion, but record the experience of one technically trained researcher and are not a substitute for the originally proposed developer-experience study, which should be conducted.

3.8 Chapter Summary

In this chapter, the Design Science Research methodology followed in this study has been outlined, the common test artefact, OWASP WebGoat, has been introduced, and the design of both the open-source and Azure cloud-native security stacks, together with the six evaluation metrics have been described. Implementation of both stacks, a comparison of the results across all six metrics, and conclusions from those results are reported in the following chapters.

4. Implementation

This chapter reports on the construction and validation of two tool stacks used in this study: one is an open source tool stack (Semgrep, Trivy, Trufflehog, Falco, OPA) and the other is an Azure cloud-native tool stack (GitHub Advanced Security/CodeQL, Dependabot, Microsoft Defender for Cloud, Microsoft Sentinel, Azure Policy), both operating on a common test artefact OWASP WebGoat. The report for each section provides information on what was constructed, the actual results of the detection, and any reported implementation challenges that arose and were addressed, as these challenges can also be evidence for metric 3.5, Setup Complexity, reported in Chapter 5. This chapter is based on the architecture of the system built in the following order: Environment provisioning, building open-source stack, building Azure stack, and finally, integration of both stacks with working CI/CD pipelines.

4.1 Environment Setup

VirtualBox was chosen over Windows to provision a dedicated virtual machine running the Ubuntu OS version 24.04.3 LTS, due to the open source falco stack, which provides the runtime anomaly detection component, requiring direct access to the Linux kernel. A Linux environment also ensured that the local testing environment would be the same as the Ubuntu-based GitHub Actions runners would be used for later on when running the pipeline, minimizing the potential for environment-specific differences between manual results and pipeline results. Instead of SSH access being set up within the VirtualBox console window, it was configured to work from the standard Ubuntu terminal, and Docker Engine was installed from Docker's official APT repository (which is not the default repository provided by Ubuntu). WebGoat was validated with a browser on port 8080 as a Docker container. During VM provisioning, one short-term problem was noticed: two kernel soft-lockup warnings were generated during the unattended operating system installation, self-recovered and never reoccurred. Prior to installing any security tooling, Docker was installed, containers were started and WebGoat's dashboard was confirmed to operate correctly to provide a clean baseline environment for testing security tools.

4.2 Open-Source Stack Implementation

The open source stack consists of five tools that each target a different layer within the software supply chain: Semgrep (static code analysis), Trivy (dependency/container image scanning), Trufflehog (secret detection), Falco (runtime anomaly detection), and Open Policy Agent

(Enforcing policies as code). All five were installed and then output was cross-validated with WebGoat, and then wired into an automated CI/CD pipeline.

4.2.1 Semgrep

Semgrep was installed using pip, and then run against WebGoat's full source tree with the free Community ruleset (160 of 544 rules applicable; 1,803 Pro rules not free, data point for Cost metric). It found 20 vulnerabilities in 974 git tracked files including SQL injection (9), path traversal, insecure cryptography (MD5 usage), server side request forgery, open redirect, Spring Actuator misconfiguration, and — significantly — shell injection in WebGoat's own release workflow on GitHub, directly related to the security aspect of CI/CD that this study addresses. There were three large, third-party JavaScript libraries, which caused scan timeouts and prevented them from being subject to full rule coverage. The Pipeline Overhead metric was the first data point, local scan time, with 78 seconds across 974 files.

```

rakesh@devsecops-research-vm: ~/WebGoat
1.167.0
rakesh@devsecops-research-vm:~/WebGoat$ semgrep --config=p/owasp-top-ten --json --output=semgrep-results.json

Semgrep CLI

i. Loading rules from registry...
Scanning 973 files (only git-tracked) with 544 Code rules:

CODE RULES


| Language    | Rules | Files | Origin    | Rules |
|-------------|-------|-------|-----------|-------|
| <multilang> | 7     | 973   | Community | 544   |
| java        | 60    | 316   |           |       |
| js          | 65    | 81    |           |       |
| html        | 1     | 69    |           |       |
| yaml        | 19    | 9     |           |       |
| json        | 3     | 8     |           |       |
| bash        | 1     | 3     |           |       |
| dockerfile  | 4     | 1     |           |       |



SUPPLY CHAIN RULES

💎 Sign in with `semgrep login` and run
`semgrep ci` to find dependency vulnerabilities and
advanced cross-file findings.

PROGRESS
100% 0:01:09

Scan Summary

☑ Scan completed successfully.
• Findings: 20 (20 blocking)
• Rules run: 160
• Targets scanned: 973
• Parsed lines: ~99.9%
• Scan skipped:
  ◦ Files larger than files 1.0 MB: 2
  ◦ Files matching .semgrepignore patterns: 102
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 160 rules on 973 files: 20 findings.
💎 Missed out on 1803 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
rakesh@devsecops-research-vm:~/WebGoat$

```

Figure 4.1: Semgrep scan results against WebGoat — 20 findings across 974 files

4.2.2 Trivy

The installation was attempted via Trivy's APT repository since the official install script couldn't be completed due to DNS resolution error. The built WebGoat Docker image was too large to run without a timeout override (default is 5 minutes, increased to 15 minutes to accommodate the size of the built image), and severity was set to CRITICAL and HIGH. It found 62 vulnerabilities: 11 vulnerabilities in the Ubuntu 24.04 OS layer and 51 vulnerabilities in WebGoat's Java dependencies, including five CRITICAL vulnerabilities, with the most

critical being CVE-2013-7285, the remote code execution vulnerability in XStream 1.4.5, deserialization with insecure deserialization. As expected, Semgrep and Trivy did not find any common vulnerability class, meaning they are complementary and not redundant: Semgrep will scan the source code and Trivy will scan the built dependency and OS layer. This distinction is relevant methodologically when comparing raw findings from tools working at a variety of layers in the software supply chain, as it will not be directly comparable without this context.

```
rakesh@devsecops-research-vm:~$ trivy image webgoat/webgoat --severity CRITICAL,HIGH --timeout 15m
2026-06-22T22:46:39Z INFO [vuln] Vulnerability scanning is enabled
2026-06-22T22:46:39Z INFO [secret] Secret scanning is enabled
2026-06-22T22:46:39Z INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-06-22T22:46:39Z INFO [secret] Please see https://trivy.dev/docs/v0.71/guide/scanner/secret#recommendation for faster secret detection
2026-06-22T22:51:11Z INFO [javadb] Downloading Java DB...
2026-06-22T22:51:11Z INFO [javadb] Downloading artifact... repo="mirror.gcr.io/aquasec/trivy-java-db:1"
2026-06-22T22:51:14Z WARN [secret] The size of the scanned file is too large. It is recommended to use '--skip-files' for this file to avoid high memory consumption. file_path="home/webgoat/webgoat.jar" size (MB)=142
883.57 MiB / 883.57 MiB [-----] 100.00% 0 B p/s ETA
883.57 MiB / 883.57 MiB [-----] 100.00% 1.90 MiB p/s 7m45s
2026-06-22T22:59:01Z INFO [javadb] Artifact successfully downloaded repo="mirror.gcr.io/aquasec/trivy-java-db:1"
2026-06-22T22:59:01Z INFO [javadb] Java DB is cached for 3 days. If you want to update the database more frequently, "trivy clean --java-db" command clears the DB cache.
2026-06-22T22:59:22Z INFO Detected OS family="ubuntu" version="24.04"
2026-06-22T22:59:22Z INFO [ubuntu] Detecting vulnerabilities... os_version="24.04" pkg_num=135
2026-06-22T22:59:22Z INFO Number of language-specific files num=1
2026-06-22T22:59:22Z INFO [jar] Detecting vulnerabilities...
2026-06-22T22:59:22Z INFO Table result includes only package filenames. Use '--format json' option to get the full path to the package file.

Report Summary
```

Target	Type	Vulnerabilities	Secrets
webgoat/webgoat (ubuntu 24.04)	ubuntu	11	-
home/webgoat/webgoat.jar	jar	51	-

```
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)
```

Figure 4.2: Trivy scan results against the built WebGoat Docker image — 62 vulnerabilities, including CVE-2013-7285

4.2.3 Trufflehog

Trufflehog was installed by the official script, which initially encountered two issues: (1) permissions issue (sufficient permissions with re-run script) and (2) auto-updater issue (no update flag to resolve). Scanning the WebGoat source tree directly led to two discoveries: both JWT tokens were embedded in WebGoat's own JWT teaching documentation, and not leaked operational credentials, making WebGoat not enough of a test case to test secret-detection capability. A controlled test was therefore built, with four intentionally created secrets (a GitHub token, an Azure Storage connection string, a database password, and a Slack webhook) placed in a separate test file that was committed to the repository so that it can be reproduced. Trufflehog successfully found two of four (50%): the GitHub token and the Slack webhook,

while the Azure Storage connection string and the plain database password were not found. This gap is large when compared with the cloud-native stack that is implemented via Azure in this study which is also revisited in the SME-suitability synthesis of Chapter 5.

The result for the deliberately planted test which was successfully controlled by the truffle hound is shown in the figure above. The result for the deliberately planted test which was successfully controlled by the truffle hound is shown in the figure .

```
rakesh@devsecops-research-vm:~$ echo "=== Planted secrets ===" && cat /tmp/test-secrets-controlled/fake-credentials.env
echo ""
echo "=== Detected (2 of 4) ===" && cat /tmp/trufflehog-controlled-test-v2.json | grep -o '"DetectorName": "[^"]*"'
=== Planted secrets ===
GITHUB_TOKEN=ghp_16C7e42F292c6912E7710c838347Ae178B4a
AZURE_STORAGE_CONNECTION_STRING=DefaultEndpointsProtocol=https;AccountName=fakestorageacct;AccountKey=aB3dEfGh1J2kLmNoPqRsTuVwXyZ0123456789ABCDEFGHIJKLMN0PQRSTUVWXYZabcdefghijklmnopqrstuvwxyz. net
DB_PASSWORD=SuperSecret123!
SLACK_WEBHOOK=https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXXXXXXXXXXXXX

=== Detected (2 of 4) ===
"DetectorName": "SlackWebhook"
"DetectorName": "Github"
rakesh@devsecops-research-vm:~$
```

Figure 4.3: Trufflehog controlled test result — 2 of 4 deliberately planted secrets detected

4.2.4 Falco

The installation was done through the official APT repository, where a modern driver eBPF was selected instead of a kernel module, without the need for further compilation. Initially, in the first few attempts, the Falco service was active with the rules loaded and no output was obtained from the systemd journal when trying to see live alerts, although the service was taking CPU. On Falco's configuration file the output was found to be going to the system log instead of the systemd journal, thus Falco was logging properly all along but in the wrong place. After being pointed at the correct log file, a test anomaly was created by running an interactive shell within the WebGoat container and trying to access a restricted system file. Falco responded immediately to the event of the shell spawn, capturing the full forensic context of the shell spawn event, such as the process, the container ID, and the user context, and confirmed that WebGoat ran as a non-root user. This initial test was followed by two additional anomaly scenarios (specific tests), one of which provided access and one that did not, in order to assess detection of both successful and denied accesses. A direct `cat /etc/passwd` within the running container was immediately reported with the alert stating exactly what command was run (`cat /etc/passwd`), the identity of the container and the user. A second attempt to access the

more sensitive `/etc/shadow` file was denied by the container's non-root permissions, but Falco was still able to detect the suspicious activity on both access attempts, with or without success in actually being able to read the file, showing that Falco detects suspicious activity rather than whether the access attempt ultimately fails. Because Falco runs at runtime, the overhead during monitoring was measured as roughly 46MB memory and 48 seconds of CPU time, measured over a 13 minute monitoring window, and still a very low overhead compared with the other four tools, which have a per-scan overhead.

```

root@devsecops-research-vm:~/WebGoat$ sudo tail -50 /var/log/syslog | grep -i falco
2026-06-28T23:38:25.718884+00:00 devsecops-research-vm systemd[1]: Started falcoctl-artifact-follower.service - falcoctl Artifact Follower: automatic artifacts update service.
2026-06-28T23:38:25.721521+00:00 devsecops-research-vm falco: Falco version: 0.44.1 (a88_64)
2026-06-28T23:38:25.726531+00:00 devsecops-research-vm falco: Falco initialized with configuration file:
2026-06-28T23:38:25.725548+00:00 devsecops-research-vm falco: /etc/falco/config.d/engine-kind-falcoctl.yaml | schema validation: ok
2026-06-28T23:38:25.726878+00:00 devsecops-research-vm falco: /etc/falco/config.d/falco-container-plugin.yaml | schema validation: ok
2026-06-28T23:38:25.727442+00:00 devsecops-research-vm falco: /etc/falco/falco.yaml | schema validation: ok
2026-06-28T23:38:25.728815+00:00 devsecops-research-vm falco: System info: Linux version 6.8.0-124-generic (build@upstream) (gcc-13 (Ubuntu 13.3.0-6ubuntu2-24.04.1) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42)
2026-06-28T23:38:25.741997+00:00 falco: Loaded plugin 'container@7.1' from file /usr/share/falco/plugins/libcontainer.so
2026-06-28T23:38:25.737406+00:00 devsecops-research-vm falco: [libse] container: Enabled 'podman' container engine.
2026-06-28T23:38:25.737864+00:00 devsecops-research-vm falco: [libse] container: * enabled container runtime socket at '/run/podman/podman.sock'
2026-06-28T23:38:25.737499+00:00 devsecops-research-vm falco: [libse] container: Enabled 'docker' container engine.
2026-06-28T23:38:25.737808+00:00 devsecops-research-vm falco: [libse] container: * enabled container runtime socket at '/var/run/docker.sock'
2026-06-28T23:38:25.737537+00:00 devsecops-research-vm falco: [libse] container: Enabled 'cri' container engine.
2026-06-28T23:38:25.737670+00:00 devsecops-research-vm falco: [libse] container: * enabled container runtime socket at '/run/containerd/containerd.sock'
2026-06-28T23:38:25.737841+00:00 devsecops-research-vm falco: [libse] container: * enabled container runtime socket at '/run/crio/crio.sock'
2026-06-28T23:38:25.737911+00:00 devsecops-research-vm falco: [libse] container: * enabled container runtime socket at '/run/k3s/containerd/containerd.sock'
2026-06-28T23:38:25.738124+00:00 devsecops-research-vm falco: [libse] container: Enabled 'containerd' container engine.
2026-06-28T23:38:25.738328+00:00 devsecops-research-vm falco: [libse] container: * enabled container runtime socket at '/run/host-containerd/containerd.sock'
2026-06-28T23:38:25.738421+00:00 devsecops-research-vm falco: [libse] container: Enabled 'lxc' container engine.
2026-06-28T23:38:25.739119+00:00 devsecops-research-vm falco: [libse] container: Enabled 'libvirt_lxc' container engine.
2026-06-28T23:38:25.739203+00:00 devsecops-research-vm falco: [libse] container: Enabled 'bpm' container engine.
2026-06-28T23:38:25.739725+00:00 devsecops-research-vm falco: Loading rules from
2026-06-28T23:38:26.162363+00:00 devsecops-research-vm falco: /etc/falco/falco_rules.yaml | schema validation: ok
2026-06-28T23:38:26.429214+00:00 devsecops-research-vm falco: The chosen syscall buffer dimension is: 8388608 bytes (8 MB)
2026-06-28T23:38:26.434394+00:00 devsecops-research-vm falco: Starting health webserver with threadiness 4, listening on 0.0.0.0:8765
2026-06-28T23:38:26.460547+00:00 devsecops-research-vm falco: Loaded event sources: syscall
2026-06-28T23:38:26.469105+00:00 devsecops-research-vm falco: Enabled event sources: syscall
2026-06-28T23:38:26.469750+00:00 devsecops-research-vm falco: Opening 'syscall' source with modern BPV probe.
2026-06-28T23:38:26.470459+00:00 devsecops-research-vm falco: One ring buffer every 25 Cpus.
2026-06-28T23:38:26.470995+00:00 devsecops-research-vm falco: [libse] Trying to open the right engine!
2026-06-28T23:38:26.518971+00:00 devsecops-research-vm falco: [libse] [184119] 2026-06-28 23:38:28 INFO: Creating follower
2026-06-28T23:38:26.545429+00:00 devsecops-research-vm falco: [libse] [184119] | artifact: falco-rules:5
2026-06-28T23:38:26.547786+00:00 devsecops-research-vm falco: [libse] [184119] | check every: 100mb
2026-06-28T23:38:26.548511+00:00 devsecops-research-vm falco: [libse] [184119] 2026-06-28 23:38:28 INFO: Starting follower
2026-06-28T23:38:26.598980+00:00 devsecops-research-vm falco: [libse] [184119] | artifact: gcr.io/falcosecurity/rules/falco-rules:5
2026-06-28T23:47:48.386553+00:00 devsecops-research-vm falco: [2147:28.12070566] Notice & shell was spawned in a container with an attached terminal | evt.type=process user=webgoat user_uid=1001 user_loginid=1 process=bash proc_command="/usr/bin/bash parent=containerd-shim command=bash terminal=38616 exe_flags=EXE_LMEX_LAYER container_id=d4ca551b6f66 container_name=webgoat container_image_repository=webgoat/webgoat container_image_tag=latest k8s_pod_name=k8s_ns_n-
2026-06-28T23:50:15.482784+00:00 devsecops-research-vm falco: [23:50:19.31242502] Notice & shell was spawned in a container with an attached terminal | evt.type=process user=webgoat user_uid=1001 user_loginid=1 process=bash proc_command="/usr/bin/bash parent=run command=bash terminal=38616 exe_flags=EXE_LMEX_LAYER container_id=d4ca551b6f66 container_name=webgoat container_image_repository=webgoat/webgoat container_image_tag=latest k8s_pod_name=k8s_ns_n-
root@devsecops-research-vm:~/WebGoat$

```

Figure 4.4: Falco detection of an interactive shell spawned inside the running WebGoat container, with full forensic context

4.2.5 Open Policy Agent (OPA)

OPA was installed via direct binary download. The same risk class identified by Semgrep, "untrusted GitHub context data used directly in an unsanitised shell step within a workflow file," was targeted by a custom Rego policy that Semgrep wrote. To support OPA's need for structured input, WebGoat's release workflow was transformed from a YAML file and tested against the policy, WebGoat reported the same line as vulnerable, but in a completely different way—declarative policy evaluation instead of an AST-based static analysis. The minimal workflow, with no risky pattern, returned an empty result, confirming that there was no false positive, as should the test for true-negatives. Both tools are independent and use different detection methods making this cross validation to give more weight to the underlying finding than being an artefact of the detecting tool. Unlike the other four open source tools, OPA does not include any pre-processed security ruleset in its box, which means that the real added value of using OPA will depend on the security ruleset that the adopting team can write, as is found in both the Setup Complexity and SME Suitability dimensions, discussed in Chapter

5. A few policies were then expanded upon (Section 4.4) to include three policies for three different classes of CI/CD risk, each with a confirmed true-positive and true-negative case.

```
rakesh@devsecops-research-vm:~/WebGoat$ cd ~/opa-policies
rakesh@devsecops-research-vm:~/opa-policies$ opa eval --data github-actions-security.rego --input /tmp/release-workflow.json "data.githubactions.deny" --format pretty
[
  "Job 'release' has a 'run' step using untrusted github context data directly: echo \"WEBGOAT_TAG_VERSION={{ github.ref_name }}"
]
rakesh@devsecops-research-vm:~/opa-policies$
```

Figure 4.5: OPA policy evaluation correctly flagging the untrusted-context shell-injection pattern (true positive)

```
rakesh@devsecops-research-vm:~/opa-policies$ cat > /tmp/clean-workflow.json << 'EOF'
{
  "name": "Clean Pipeline",
  "jobs": {
    "build": {
      "runs-on": "ubuntu-latest",
      "steps": [
        { "name": "Safe step", "run": "echo \"Building project\"" }
      ]
    }
  }
}
EOF
rakesh@devsecops-research-vm:~/opa-policies$ opa eval --data github-actions-security.rego --input /tmp/clean-workflow.json "data.githubactions.deny" --format pretty
[]
rakesh@devsecops-research-vm:~/opa-policies$
```

Figure 4.6: OPA policy evaluation against a clean control workflow, returning no finding (true negative)

4.3 Azure Cloud-Native Stack Implementation

The Azure stack consists of five components: Azure Policy (governance and compliance), Azure Defender for Cloud (aggregated posture and DevOps security), Dependabot (dependency scanning), GitHub Advanced Security's CodeQL (static code analysis), and Azure Sentinel (SIEM/behavioural correlation). However, because prior stages of this study had utilized a manually-cloned copy of WebGoat instead of a version-controlled submodule, and Azure's tooling for accessing Git is available only within the GitHub-native context, WebGoat was added to the repository as a submodule.

4.3.1 CodeQL (GitHub Advanced Security)

After trying a few different settings for build-mode and JDK version, a CodeQL Advanced workflow was crafted that included recursive submodule checkout, JDK 25, and a manual Maven build step. The scan found 71 findings: 3 Critical (two of which were deserialization issues, the other was an XXE injection vulnerability), 52 High (which were mostly repeated path traversal vulnerabilities, not 52 distinct vulnerability types) and 16 Medium, 11 of which were in the target application, WebGoat, and the remaining 5 were found in the project's own CI workflow configuration, in the real scan, not in WebGoat itself. Similarly to the Trivy vs. Semgrep comparison, the number of findings CodeQL produces (71) cannot be directly

compared with the number of findings Semgrep produces (20) when you don't consider differences in scan scope and rule depth of the two tools.

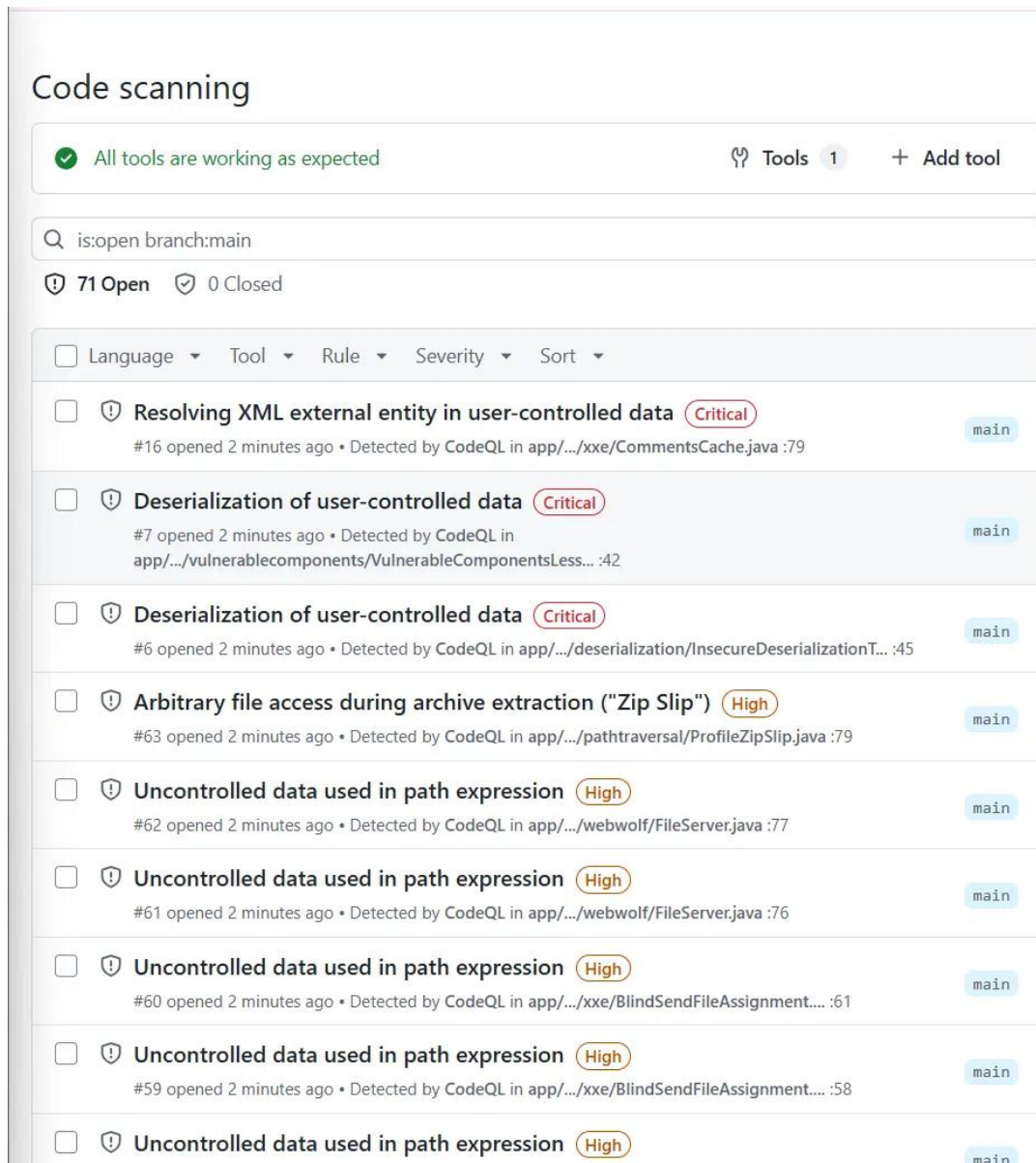


Figure 4.7: CodeQL scan results against WebGoat — 71 findings (3 Critical, 52 High, 16 Medium)

4.3.2 Dependabot

WebGoat's dependency graph and dependency alerts were set to true in repository settings but didn't trigger any alerts for WebGoat's Java/Maven dependencies. This turned out to be a platform limitation and not a lack of vulnerabilities: Dependabot does not automatically discover the contents of a Git submodule, so any dependency manifest contained in such a submodule (like the WebGoat's Maven `pom.xml`) is out of scope. Both of the above approaches were deemed to be a lesser effort for the same benefit as the Docker image-based

scan that Trivy already offers, where the Docker images are part of the upstream repository of the WebGoat project (Section 4.2.2, 62 findings). This was confirmed three ways: Trivy's scan, WebGoat's own public upstream repository (which has dependency graph enabled, and does confirm the same vulnerable version of XStream is used, with a comment to that effect in the source code), and direct source inspection. An interesting and valid finding, as a team using Dependabot only and not aware of the submodule limitation would have a false sense of dependency-vulnerability coverage for any submodule-included code, a typical situation of the team category this study focuses on.

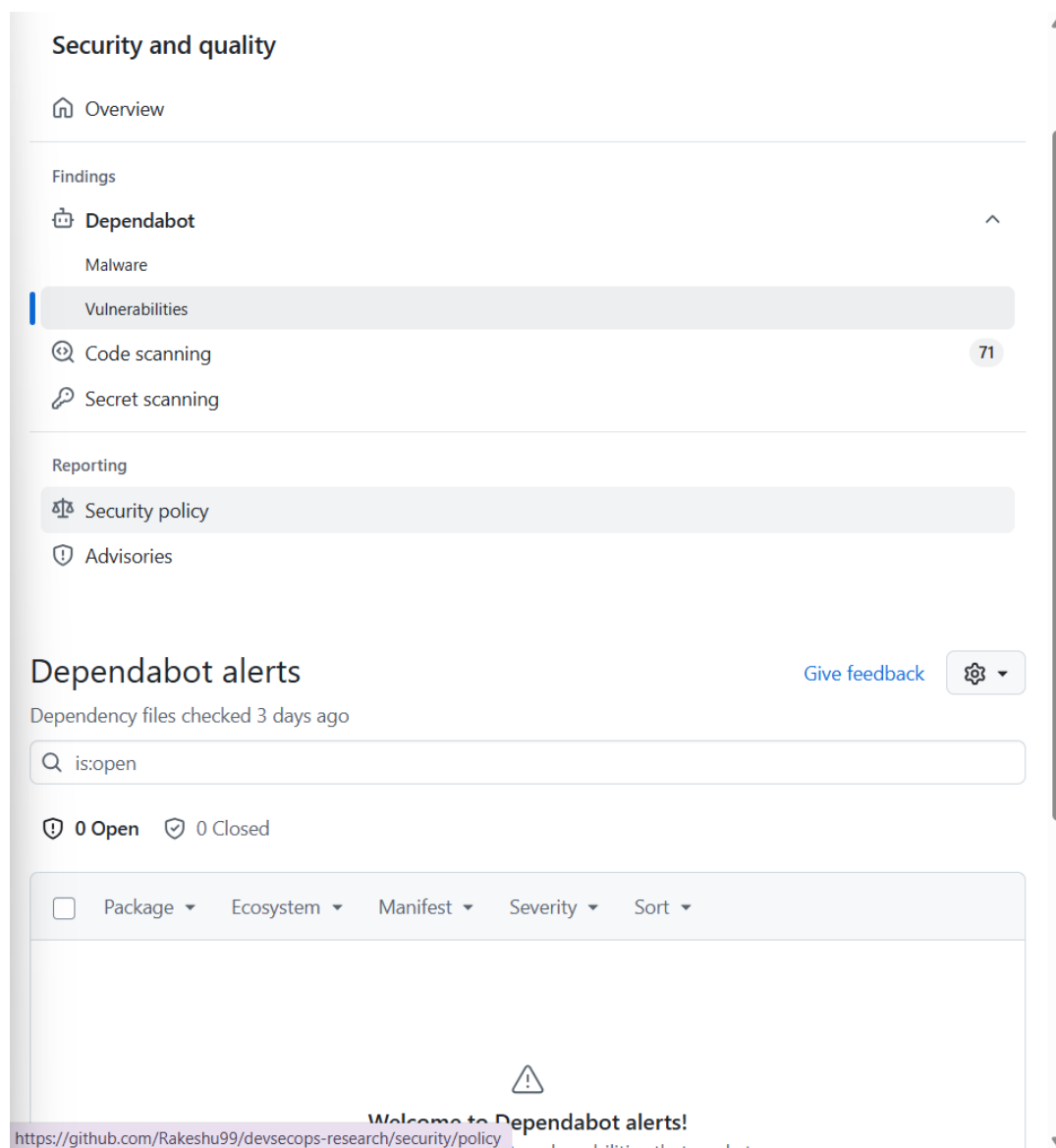


Figure 4.8: Dependabot alerts showing zero findings despite the presence of known vulnerable dependencies

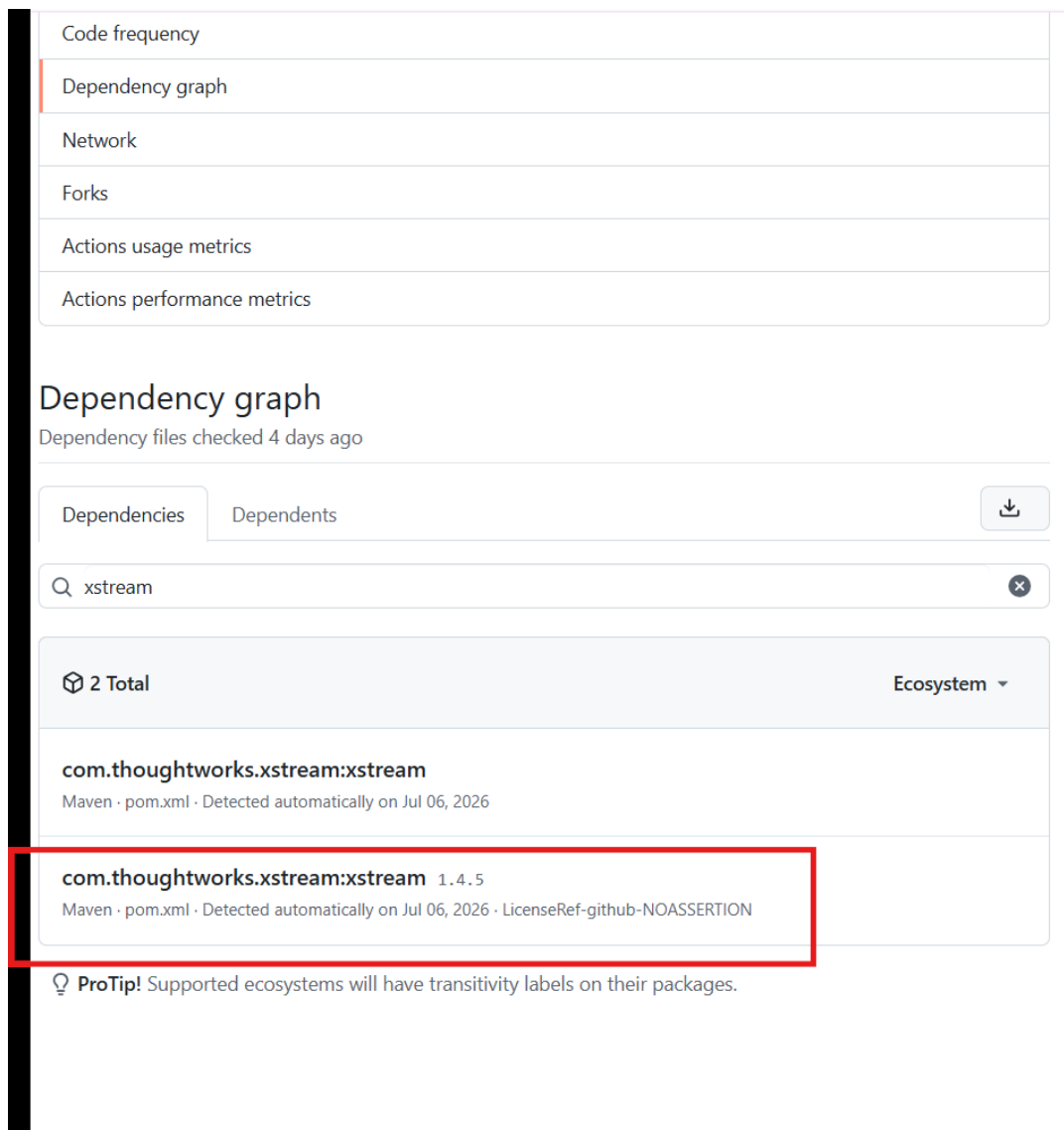


Figure 4.9: WebGoat's upstream repository confirming the same vulnerable XStream dependency, corroborating the submodule-scope limitation

4.3.3 Microsoft Defender for Cloud

A GitHub Connector was set up to connect the repository with Microsoft Defender for Cloud. The initial connector creation failed with a policy-denial error; the cause was a subscription-level policy that restricts connections from an invalid region from being created, but is not in the connector wizard location dropdown and only fails at connector creation time as shown in the Azure Activity Log raw JSON output. The connector was rebuilt with an allowed region and was successful. The DevOps Security dashboard said there were 75 total findings (3 Critical, 52 High, and 20 Medium): 71 Code findings, 4 Infrastructure-as-Code findings, and zero Secret or Dependency findings.

A comparison of the Defender's Code findings with the findings generated by CodeQL, in Section 4.3.1, reveals an exact match in the Critical and High counts, meaning that Defender for Cloud's DevOps Security dashboards surface a high percentage of the CodeQL results, but not an exact replica of the detection engine that it runs. The remaining 75 findings are pushed by the additional lightweight scanners bundled into the connector, and account for the small difference in findings between the two. This is an interesting result for the comparative analysis in Chapter 5: If an SME was already using CodeQL directly it would benefit from centralised dashboarding from Defender for Cloud, but not as much in terms of security posture. When working within Defender for Cloud, two additional cross-portal inconsistencies were detected.

First, the general Overview blade that is not the DevOps Security blade described above reported 100% secure score and 0 findings, while the DevOps Security blade reported 75 findings and a 31.2% secure score; when you went directly between the two blades within a session, both reported were the same, so the two blades were confirmed to be very different views rather than a stale cache. Second, the Cloud Security Overview page includes a filter that modifies the figures it reports on without the need to navigate anywhere: When the filter is set to off, it reports 31.2%, "At risk", 8 assets, and when the filter is turned to on and the scope is narrowed to Azure, it reports 86.9%, "Good", 1 asset, and on the same page. This second is a beefed-up version of the cross-blade or cross-portal discrepancy, as it shows the same page with two materially distinct security postures on the same environment when controlled by a filter state that could easily be overlooked.

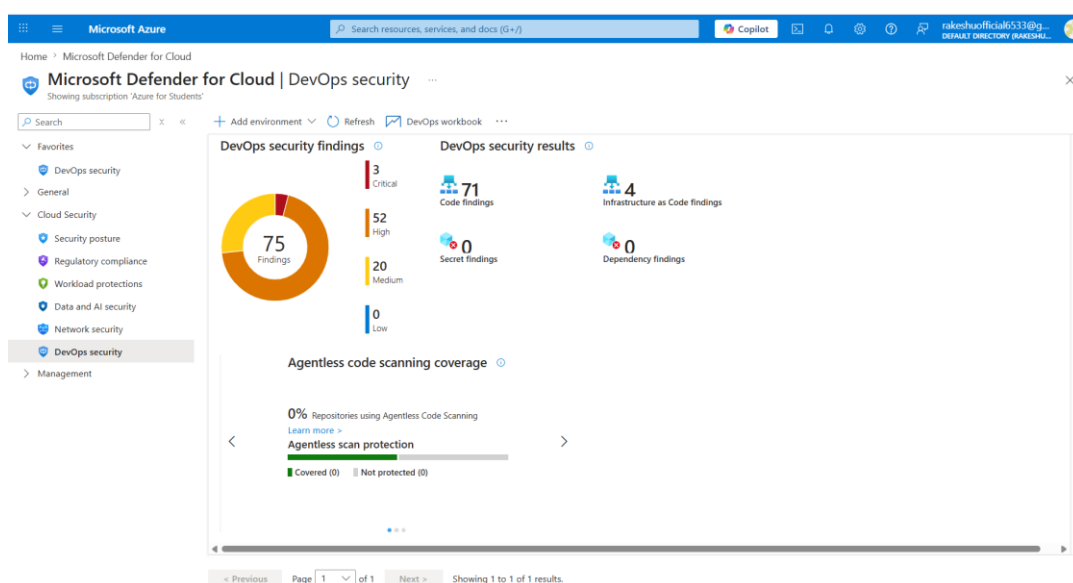


Figure 4.10: Microsoft Defender for Cloud DevOps Security overview — 75 findings

4.3.4 Microsoft Sentinel

A log analytics workspace was created and Sentinel was onboarded to it. Even though Defender for Cloud reported 75 findings, all of the Sentinel Incident and Alert views were still empty. It was not a connector failure, as the tenant had not gone through a separate and obscure onboarding step (Cloud Security tenant preparation) from the Defender for Cloud GitHub connector and Sentinel's data connector gallery. The preparation step was undertaken and, after the multi-hour completion window, the Cloud Security Overview accurately started reporting a connected environment, populated security posture score and found assets. After this fix, however, there were still no incidents or Alerts. The true culprit was found after further investigation—posture and configuration Recommendations instead of active threat Security Alerts from Defender for Cloud. Recommendations are expected behaviour and not Incidents, they are fed into Secure Score and Posture dashboards by architecture.

The initial posturing finding that should emerge as Incidents was in itself the mistake. This difference – and the lack of clarity regarding this difference at initial setup – is a real SME-suitability finding: a team might reasonably think that its SIEM integration has failed when it actually is working perfectly as designed. One other difference was noticed during this diagnosis: Defender's own portal and the Azure-native Sentinel blade, both having the same workspace, reported different numbers of connectors (1 versus 7), even though Microsoft indicated that it is working on integrating the UI between the two products (Figures 4.11 and 4.12). This was directly communicated to Microsoft support, who confirmed that this was an older view of the portal and it was only now being updated to the correct backend connection state, citing that this was the ongoing process of migrating to a new platform as reported in the Microsoft's published guidance for the transition from grouped to individual security recommendations (Microsoft, 2026d) in a related section of the product. So the actual figure is considered the seven in the back. This finding is still valuable SME-suitability evidence: if an SME hadn't been able to find out that the bottom number was "old" instead of "new," then he or she might have thought that both numbers were correct.

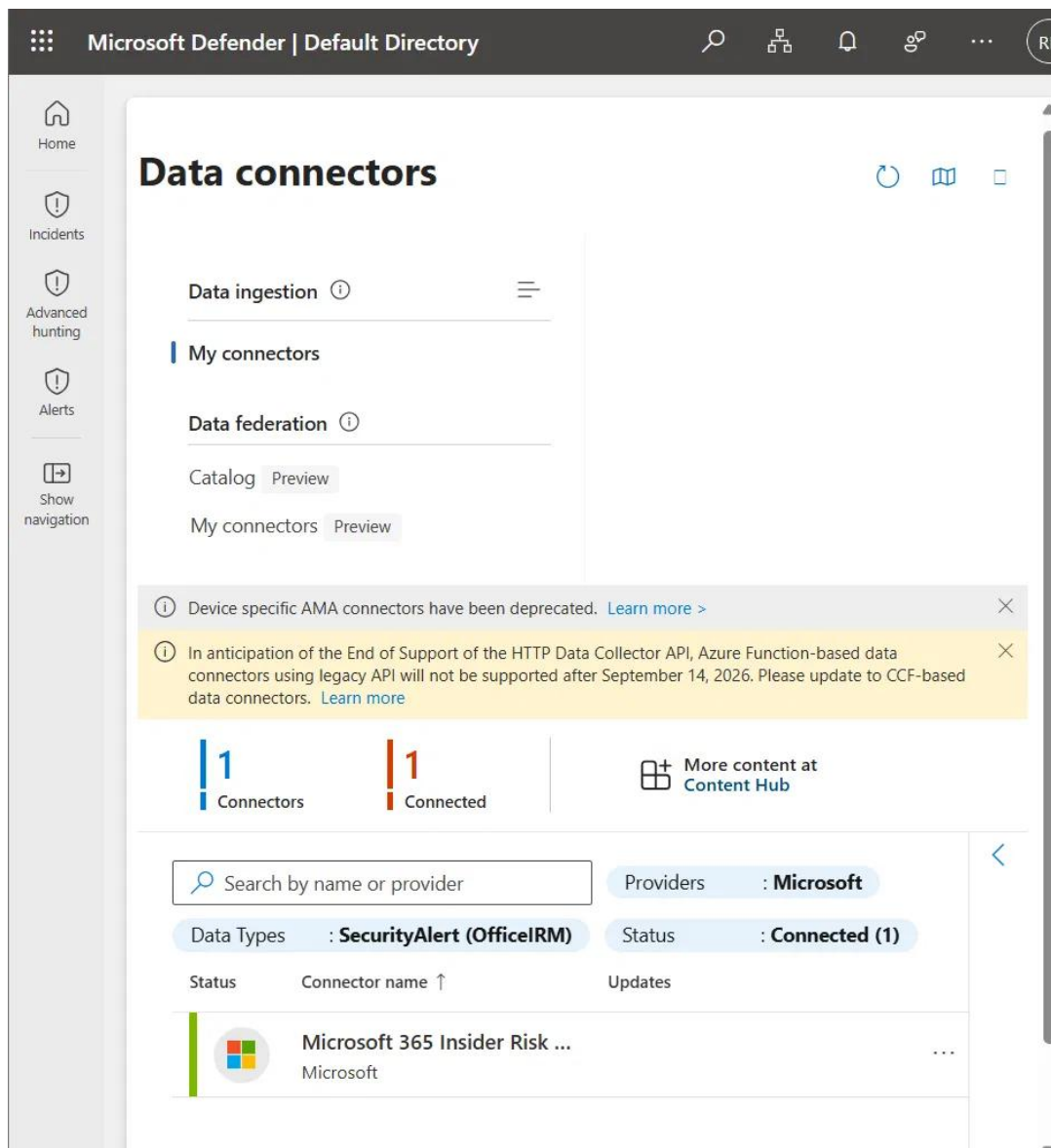


Figure 4.11: Microsoft Defender portal showing 1 connector, 1 connected, for the devsecops-research-law workspace

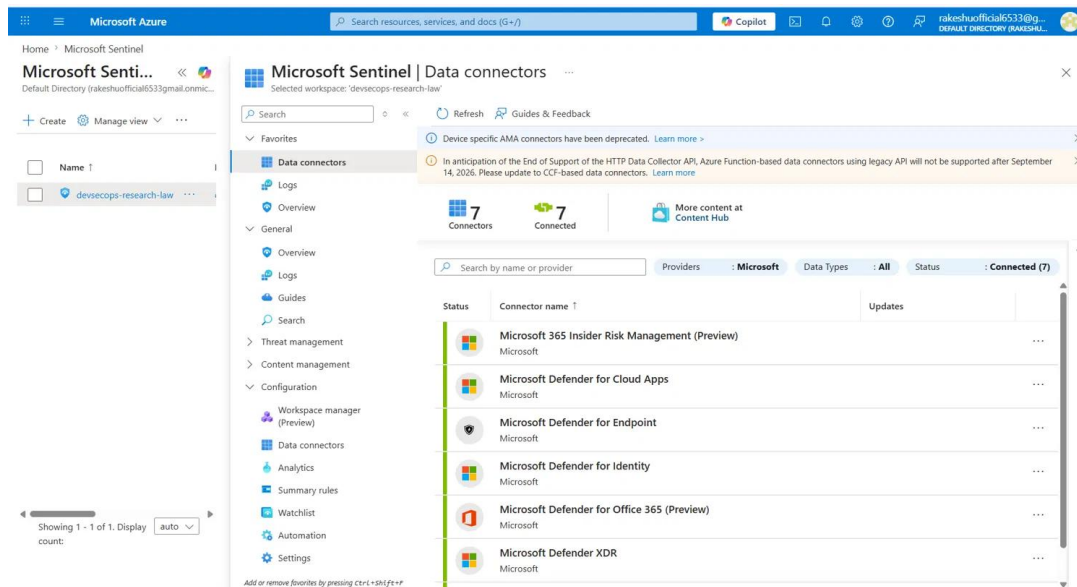


Figure 4.12: Microsoft Sentinel's own Azure-native blade, same workspace, showing 7 connectors, 7 connected

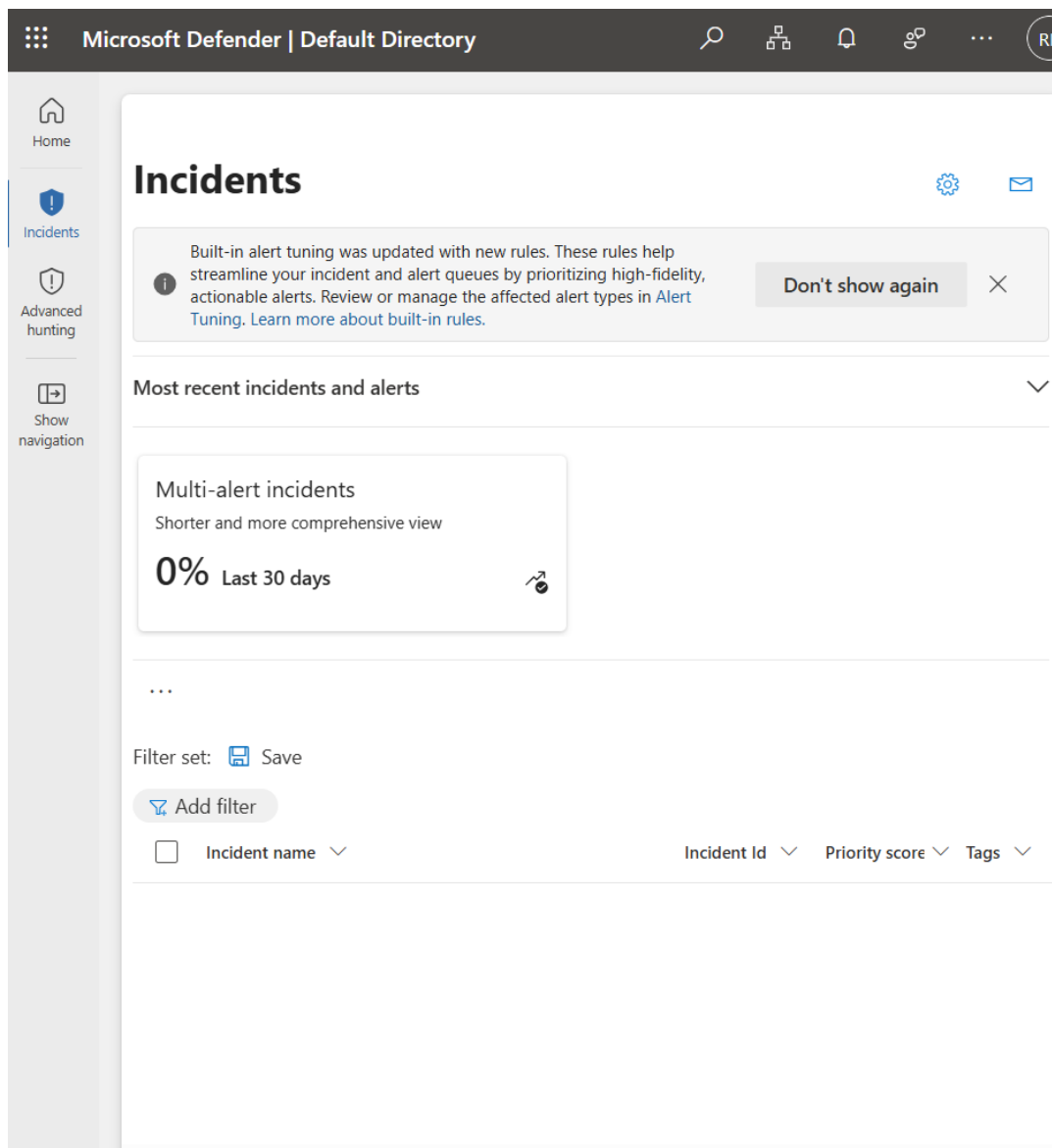


Figure 4.13: Microsoft Sentinel Incidents view showing zero incidents despite 75 confirmed Defender for Cloud findings

4.3.5 Azure Policy

An existing default policy assignment ("ASC Default", 226 policies, Audit effect) was found to be auto-provisioned on subscription creation, without requiring any manual configuration. The 'Microsoft cloud security benchmark v2' initiative was also added at subscription scope for full coverage of this baseline policy set (730 policies, 92 control groups). After evaluation, compliance results were checked via two portals and found to report six different, non-reconciled metrics that are reasonable for the reader to expect to be a single compliance figure: overall resource compliance (25%), individual policy-check pass rate (approximately 95.4%), initiatives fully compliant (0 of 2), and two different regulatory compliance percentages from the Defender for Cloud own reporting (50% and 56.25% for the two different assigned

initiatives), a separate non-compliant-resource count from the remediation view, and a sixth figure from the Regulatory Compliance page itself reporting compliance as a category-level rollup ("56 of 63 controls passed", approximately 89%), a materially different unit of aggregation from the other five.

This rollup is directly relevant to this study's argument on security posture because the four categories listed as failing to meet the requirements in the rollup are Privileged Access, Data Protection, Logging and Threat Detection, and Incident Response. That's a stand-alone Setting Complexity and SME-suitability finding, though: Azure Policy and Defender for Cloud don't provide a single authoritative compliance percentage, and simply referring to "compliance" without stating the specific one of six compliance metrics you're referring to is easy to get wrong for a team not familiar with the platform.

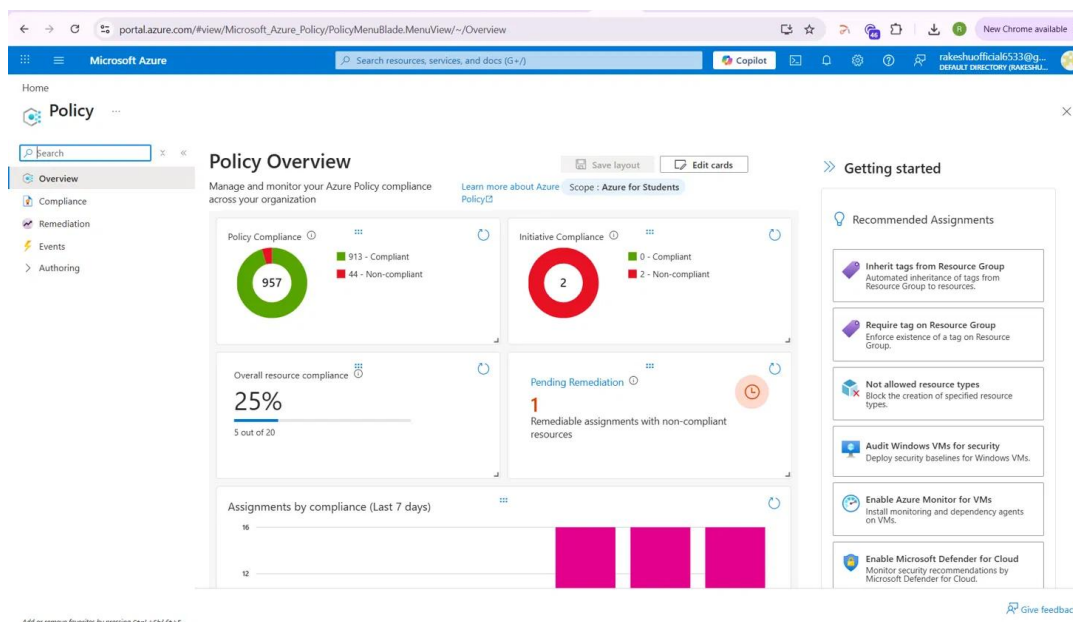


Figure 4.14: Azure Policy compliance overview showing multiple, non-reconciled compliance metrics

4.4 Pipeline Integration and Verification

The two stacks were embedded in automated CI/CD pipelines on GitHub, along with a baseline (without any security tooling, checkout and simulated build only), so that a controlled comparison of the timing could be done. The problem was discovered during verifying the open-source stack's pipeline, and a major issue was identified: the pipeline's checkout steps would never fetch sub-modules, so every automated check performed before this issue was resolved against an empty repository – even though there were checks performed manually, using a local copy of WebGoat that had valid results. Two additional tool-specific issues that

were not detected until WebGoat was actually deployed for the pipeline to scan were revealed by the checkout step: Semgrep's default file discovery method was based on the git-tracked-file status which is not satisfied by the content of submodules, so an additional flag was needed to enable filesystem-based discovery; and Trivy's filesystem-scan mode attempted to perform live Maven dependency resolution against Maven Central, but failed due to the shared IP rate limits of the GitHub-hosted runners, requiring a switch to image-based scanning consistent with the method already verified manually in Section 4.2.2. These changes were made – and a new pipeline run was compared to each tool's previously reported manual findings to ensure that finding categories and file paths matched. This cycle of build, evaluate, discover flaw, refine, re-evaluate is considered in this study as a legitimate Design Science Research iteration for the evaluation instrument and is separately citable as a legitimate finding of the Setup Complexity: correctly configuring the open-source tools to be incorporated into a real pipeline, and against a codebase of Git submodules, required non-obvious tools-specific configuration beyond a basic checkout step.

Only the Azure stack's pipeline was run for CodeQL. The reason is, it's not a mistake: Dependabot, Defender for Cloud, Sentinel and Azure Policy are not inline CI steps, they are evaluated asynchronously at the platform level, outside the window of the pipeline. This is one of the architectural differences itself, which is a key finding for the Pipeline Overhead metric, documented in full in Chapter 5.

Each pipeline (baseline, open-source, Azure) were triggered manually, back-to-back, with the same type of GitHub-hosted runner under similar conditions, and was then repeated across three interleaved trials to measure within-study stability and minimize the need for any single-run observation. The mean times for the baseline pipeline, open-source stack and Azure stack (CodeQL only) were 16.3 seconds, 65.7 seconds and 157.7 seconds respectively, with a low variance (3 to 6 seconds spread) among the three repeated trials for each of the stacks. A large portion of this difference is due to the fact that CodeQL must build WebGoat using Maven before analysis can be performed, while the other four open source tools do not perform a build step: Semgrep operates directly against source or a pre-built container image, and the other two, Trufflehog and OPA, have no build step. The baseline pipeline does not compile, it actually runs a simulated build step (a fixed delay), so the differences in reported ratios are not because one pipeline or the other took a different amount of time preparing the build to perform the security analysis step, but because they each have a different amount of security analysis

time as part of the process. This is not what this study interprets as a “slow down” of the Azure stack, but rather as a legitimate and architectural depth versus speed choice of the Azure stack's compiled data-flow aware analysis over the lighter weight analysis methods of the open-source tools.

4.5 Chapter Summary

The five component security stacks were configured and tested in the validation and comparability boundaries described in Chapter 3, and validated against their manual security stack validation results, and integrated into working and automated CI/CD pipelines. For the ten individual tools and platform components reported in this chapter, real results were produced on a common test artefact, and a number of genuine implementation flaws were identified, understood and fixed during this process: the pipeline submodule bug, the Sentinel tenant-preparation gap, the Azure region-restriction policy failure, all of which provided citable evidence directly relevant to the metrics of Setup Complexity and SME Suitability. Based on the results presented here, the six-metric comparative analysis at the heart of this study's research question is performed in chapter 5.

5. Comparative Analysis

The open-source and Azure cloud-native stacks presented in Chapter 4 are compared in this chapter on the basis of six evaluation metrics: detection capability, false positive rate, complexity of installation, overhead of the pipeline, cost, and suitability for SMEs. In keeping with this study's DS methodology and commitment to empirically supported conclusions, each metric is presented with the data used to compute it, prior to the statement of the finding.

5.1 Detection Capability

	Open-source stack	Azure stack
Code-level SAST	<u>Semgrep</u> : 20 findings	<u>CodeQL</u> : 71 findings (3 Critical, 52 High, 16 Medium)
Dependency/container scanning	<u>Trivy</u> : 62 findings	<u>Dependabot</u> : 0 (submodule limitation; not an absence of vulnerabilities — see Section 4.3.2)
Secret detection	<u>Trufflehog</u> : 2/4 (50%) in controlled test	No direct Azure equivalent tested
Runtime detection	Falco: shell-spawn detected, full forensic context	Sentinel's behavioural layer not exercised (Section 6.2, Future Work)
Policy enforcement	OPA: 3 policies, 6/6 true-positive/negative cases confirmed	Azure Policy: 226 + 730 policies evaluating (Section 5.6)
Aggregated posture	—	Defender for Cloud: 75 findings, confirmed to substantially mirror <u>CodeQL</u> (Section 4.3.3)

Table 5.1: Detection capability by layer, open-source vs Azure stack

Finding count is NOT directly comparable across stacks. Defender for Cloud's 75 findings are not independent of the findings identified by CodeQL's 71 (Section 4.3.3 establishes an exact match on Critical and High counts) and Dependabot's 0 findings are a platform limitation, but also a reflection of the fact that vulnerabilities were discovered by Trivy in the same codebase. The methodologically valid and appropriate comparison is made between tools on the same level of the software supply chain: code level (Figures 4.1 and 4.7), and the number of findings produced by CodeQL comes at a higher cost of depth and a lower cost of speed, consistent with the patterns observed in Section 4.4.

Table 5.1a clarifies the comparability boundary in this section for all six evaluated layers, as opposed to stating it in prose. The two stacks are not directly scored against each other in this study for those layers that are not marked as directly comparable, but are described separately.

Layer	Open-source	Azure	Comparability	Reason
SAST	<u>Semgrep</u>	<u>CodeQL</u>	Direct	Both analyse source code statically
Dependency scanning	<u>Trivy</u>	<u>Dependabot</u>	Partial	Different discovery mechanisms; <u>Dependabot</u> has a submodule scope limitation (Section 4.3.2)
Secret detection	<u>Trufflehog</u>	None tested	Not comparable	No Azure-native secret-scanning component was evaluated in this study
Runtime detection	Falco	Sentinel	Not comparable	Falco was successfully tested across three separate anomaly scenarios (Section 4.2.4); Sentinel's behavioural layer had no adversarial telemetry available to test against (Section 4.3.4) — this is an asymmetry in evidence, not a mutual absence of testing
Policy enforcement	OPA	Azure Policy	Partial	Different policy models and enforcement points
Posture aggregation	None	Defender for Cloud	Not comparable	Azure-specific aggregation layer with no open-source equivalent evaluated

Table 5.1a: Comparability of evaluated controls by layer

This imbalance in the amount of work needed to get a runtime detection layer to a testable state can itself be a useful observation, regardless of any completed behavioural test. Using infrastructure already in this study's environment, Falco was installed, tested, and successfully validated in a single local install with real, verified detections in three different scenarios: a shell-spawn event, a successful `/etc/passwd` read, and a denied `/etc/shadow` attempt that was correctly flagged. To meaningfully test any adversarial simulation against Sentinel's equivalent behaviour capability, it would have required a baseline of ambient behaviour to be established over a span of days, with an Azure VM to be provisioned and a monitoring agent to be installed. None of these were within the scope of this study. The lack of anything more than a few clicks to get from a tool to a testable state is directly relevant to SME Suitability (Section 5.6): a resource-constrained team can generate real, working runtime detection evidence in an afternoon using Falco, while the equivalent evidence from Sentinel requires infrastructure investment that many of the same teams would not readily be available.

5.2 False Positive Rate

Policy	Risk class	True Positive	True Negative
Shell injection	CICD-SEC-1: Insufficient Flow Control	Confirmed (1 finding)	Confirmed
Action pinning	CICD-SEC-3: Dependency Chain Abuse	Confirmed (7 findings)	Confirmed
Secret logging	CICD-SEC-6: Insufficient Credential Hygiene	Confirmed	Confirmed (0 findings)

Table 5.2: OPA policy evidence base — three risk classes, six ground-truth cases

This study's false positive evidence is scoped to controls where paired ground-truth test cases could be constructed, instead of a stack-wide false positive rate across all ten tools evaluated, and is drawn from OPA's expanded policy set: three distinct CI/CD risk classes, each independently validated against a true-positive case (a real vulnerable pattern correctly flagged) and a true-negative case (a clean control correctly returning no finding), for six total ground-truth data points (see Figures 4.5 and 4.6 for a representative true-positive/true-negative pair). It follows the methodological approach reported by Aloraini et al. (2019) that show false-positive behaviour is systematic based on the type of finding, not uniform on all the output of the SAST tool, thus motivating the study of each type of finding separately. While developing this evidence base, two real implementation bugs were identified and fixed: one was an OPA CI job evaluating its own workflow file, rather than WebGoat's, that caused no error to be displayed, but only became apparent during unrelated verification tasks; and another was a false positive that was introduced in an early version of the action-pinning policy, documented in Section 4.2.5, and therefore relevant evidence for Setup Complexity. There were three positive controls and all of them were detected; three negative controls were tested, but none resulted in a finding, hence, no false positive has been observed in the negative controls tested. These results are not indicative of a false-positive rate, and should not be taken as a generalisation of the results or tools analyzed in this study.

5.3 Setup Complexity

Friction point	Stack	Diagnosis difficulty
CI submodule checkout bug	Open-source	Medium — found via cross-checking CI artifacts against manual results
OPA CI job targeting wrong file	Open-source	High — silent, no error, undiscovered until unrelated work exposed it
Region-restriction policy not reflected in connector UI	Azure	High — required raw Activity Log JSON inspection
Sentinel zero-incidents (tenant onboarding gap)	Azure	High — multi-step diagnostic path
Dependabot submodule gap	Azure	Medium — required corroboration against upstream repository
Cross-portal inconsistency (5 instances)	Azure	Medium — requires knowing to check more than one view or filter state
Six non-reconciled compliance metrics	Azure	High — each individual metric was easy to locate, but determining which of six differently-scoped figures was authoritative required cross-referencing multiple portals and rollout levels

Table 5.3: Documented friction points by stack

During implementation, the open-source stack identified two friction points, while the Azure stack identified six, and the Azure-stack friction points were in general more difficult to diagnose – three of them required examining the raw stack logs, multi-portal cross-referencing (see the direct example below in Figures 4.11 and 4.12) or long wait times during onboarding as opposed to being directly visible from the tool or platform's UI. The corresponding cases reported in Chapter 4 are concrete, first hand accounts of the overall pattern reported by Rajapakse et al. in their literature-wide systematic review in 2022.

5.4 Pipeline Overhead

Round	Baseline	Open-source	Azure (CodeQL)
1	18s	1m 4s (64s)	2m 34s (154s)
2	16s	1m 7s (67s)	2m 40s (160s)
3	15s	1m 6s (66s)	2m 39s (159s)
Mean	16.3s	65.7s	157.7s
Range	15–18s	64–67s	154–160s

Table 5.4: Pipeline execution time, three interleaved trials, 16 July 2026

Within-study variation (variance) was low and similar for all three interleaved trials (3–6 second spread per pipeline), which indicates low within-study variation and not artefacts in single runs. For the open-source stack, the duration was multiplied by 3.5 to reach the baseline duration; for the Azure stack, the baseline duration was multiplied by 9.7. Unlike the four open-source tools, this gap is not incidental but architecturally explained as CodeQL's manual build mode necessitates compiling WebGoat prior to analysis. Critically, Dependabot, Defender for Cloud, Sentinel, and Azure Policy do not add any measurable pipeline overhead as they all evaluate at the platform level asynchronously and not as CI steps, so the 157.7 seconds Azure number is only for the CodeQL component and doesn't include the other four components of the Azure stack.

5.5 Cost

	Open-source stack	Azure stack
Licensing	Free / open-source at all usage levels tested	CodeQL and Azure Policy free; Defender for Cloud and Sentinel on 30– 31 day trials, per-resource/per-GB billing thereafter
This study's actual exposure	€0	€0, confirmed via Azure's own cost estimate showing \$0.00 (0 GB ingested across the trial period)
Trial expiry	N/A	9–10 August 2026 (calculated from the 10 July start date; not directly surfaced anywhere in the Azure portal UI)

Table 5.5: Cost comparison, open-source vs Azure stack

All the open-source components tested in this study have no recurring software licensing fees (under the usage conditions assessed) beyond any compute time, CI minutes or researcher effort that the stack used. The cost of the Azure stack was also observed to be €0 in this study, but only because Defender for Cloud and Sentinel ran for a limited time in a free trial window; this study does not claim to measure the costs of the Azure stack once it moves out of the free trial window, which is a limitation as noted in Section 6.2.

5.6 SME Suitability

Dimension	Favours
Ongoing cost	Open-source
Detection transparency (traceable to a specific rule or pattern)	Open-source
Initial setup friction	Open-source (2 friction points vs Azure's 6, several of which were harder to diagnose)
Ongoing interpretive complexity	Open-source (cross-portal inconsistency observed 5 times; 6 non-reconciled compliance metrics)
Detection depth (semantic, data-flow-aware analysis)	Azure (CodeQL)
Pipeline speed	Open-source

Table 5.6: SME suitability synthesis across all preceding metrics

SME Suitability is not independently measured but a composite of the five metrics listed above, in keeping with methodology identified in Section 3.5. This study concludes that the open source stack is more transparent and not expensive, and, specifically in CodeQL, that its detection depth is less than that of the Azure stack for an SME that doesn't have its own security engineer. While this cross-portal inconsistency pattern in Sentinel, Azure Policy, and Defender for Cloud (Sentinel's connector count, Section 4.3.4; Azure Policy's six non-reconciled compliance metrics, Section 4.3.5; Defender for Cloud's general Overview blade vs. its DevOps Security blade, Section 4.3.3; and the Cloud Security Overview page's environment-filter toggle, Section 4.3.3) is treated as a standalone SME-suitability finding instead of as a peripheral usability complaint. Of the five such cases, the Sentinel-versus-Defender connector count was later determined to be caused by a particular (currently in use) platform migration (Section 4.3.4) and the remaining four cases are not explained in Microsoft's public documentation and cannot be addressed within the scope of this study. The difference is important: Azure's tools are individually powerful, but Azure interfaces as seen in this study didn't provide a consistent interpretation of security state across Azure portals, and that is an important difference in the case of a team that may not have the platform knowledge to determine which view is authoritative.

5.7 Synthesis Against the Study's Hypotheses

We put two hypotheses to the test in Section 1.2. H1 forecast a lower cost for the open-source stack, compared to the Azure stack, as well as lower pipeline overhead and lower setup complexity. Based on the data directly reported in Sections 5.3 and 5.4, the number of friction points is two versus six, and the pipeline overhead is 65.7 seconds versus 157.7 seconds, for H1. The cost component could not be fully validated experimentally because the Azure evaluation happened within the borders of a free trial-access (Section 5.5); the cost finding is

thus limited to the structure of licensing and to the expenditure in the trial period and not to the costs of operating an SME. H2 said that the detection capability would not vary significantly between the two stacks, because the core static analysis components of both stacks would work on the same structural-analysis principles, and that any Azure advantage would rely on the "behavioural and anomaly detection" capabilities of the stack, which are powered by AI and ML. As originally stated, the hypothesis was that CodeQL would report a narrower set of structural findings than Semgrep under the configurations evaluated. This is not the case for CodeQL; under the configurations evaluated, CodeQL reported a much wider set of structural detection coverage than Semgrep. By contrast, the AI/ML specific behavioural component of H2 was not evaluated, but not necessarily proven to be wrong, because, during the period of this study (Section 4.3.4), Sentinel's behavioural correlation engine did not have any adversarial telemetry to evaluate. The difference between a hypothesis that is disconfirmed and a hypothesis that is untested, is not the negative finding about AI/ML-augmented tooling that is typically found in a conclusion, but rather is explicitly stated in section 6.2 as an important limitation of this study. The lack of a proven edge does not mean that AI/ML behavioural detection is ineffective, just that it was not demonstrated under the adversarial telemetry conditions that were not considered in this study.

Table 5.7 provides a single, easy-to-understand synthesis of the research question, the two hypotheses, and the six metrics, which were each discussed above.

Research Element	Finding
Research Question	Can a lightweight AI/ML pre-pipeline security gate be deployed in a DevSecOps CI/CD pipeline in a way that measurably improves security outcomes for SMEs without adding significant complexity or operational overhead?
H1	Supported for setup complexity and pipeline overhead; cost conclusion limited to trial-period evidence, not sustained SME operating cost
H2	Structural detection advantage demonstrated for CodeQL; AI/ML-specific behavioural advantage untested, not disproven
Detection Capability	Under the configurations evaluated, CodeQL produced substantially broader structural finding coverage than Semgrep (71 vs 20 findings), at the cost of build-step dependency and longer pipeline execution
False Positive Rate	3/3 positive controls detected, 0/3 negative controls produced a false positive; not a statistically generalisable, stack-wide claim
Pipeline Overhead	Azure (CodeQL) adds 9.7× baseline execution time versus the open-source stack's 3.5×
SME Suitability	Open-source stack favoured where cost certainty, transparency, and lower operational burden outweigh raw detection depth
Key Limitation	Sentinel's behavioural telemetry was unavailable within this study's scope; no claim is made about behavioural detection effectiveness, only that it remains untested

Table 5.7: Research question, hypothesis, and metric synthesis

Table 5.8 is a summary of both hypotheses and their outcomes in a single reference view, which complements the synthesis of research questions and metrics presented in Table 5.7.

Hypothesis	Prediction	Outcome	Evidence
H1	Open-source stack demonstrates lower setup complexity, lower pipeline overhead, and lower cost than the Azure stack	Supported for setup complexity and pipeline overhead; cost component supported only at the level of licensing and cost certainty, not sustained SME operating cost	Sections 5.3, 5.4, 5.5
H2	Detection capability will not differ substantially for structural, signature-detectable vulnerabilities; any Azure-specific advantage depends on AI/ML-augmented behavioural detection	Not supported for structural detection: under the configurations evaluated, CodeQL produced substantially broader structural finding coverage than Semgrep, contradicting the hypothesis as stated; the AI/ML-specific behavioural component remained unevaluated rather than disproven	Sections 5.1, 5.7, 6.3

Table 5.8: Summary of hypothesis outcomes

5.8 Chapter Summary

In each of the six metrics, the costs of the Azure cloud-native stack are real, proven, and significant: six friction points compared to two, 9.7 times the overhead of the pipeline compared to 3.5 times, and materially higher interpretive complexity across the surfaces it reports. This study could not make use of the AI/ML-enhanced behavioural detection capability that the Azure stack's marketing focuses on, but the advantage the principal showed was the deep structural detection capabilities of CodeQL. The findings from this study are summarized in Chapter 6, where they are discussed in terms of their implications, significance to the existing literature, and strengths and weaknesses of this study, followed by a discussion of the contribution of this study and future work in Chapter 7.

6. Discussion

This chapter brings together the findings reported in Chapters 4 and 5 to answer the study's research question directly, rather than to muffle it and honestly identifies and states limitations of the study that it did not have the capacity to resolve; it also provides clear directions regarding future work to fill gaps that could not be addressed by this study.

6.1 Summary of Findings and Answer to the Research Question

Metric	Winner	Evidence
Detection Capability	Azure (CodeQL)	71 findings vs Semgrep's 20 (Section 5.1)
False Positive Rate	Validated method, <u>open-source</u> only	3/3 positive controls detected, 0/3 negative controls produced a false positive (Section 5.2)
Setup Complexity	<u>Open-source</u>	2 friction points vs 6, several <u>harder</u> to diagnose (Section 5.3)
Pipeline Overhead	<u>Open-source</u>	65.7s vs 157.7s — 3.5× baseline vs 9.7× (Section 5.4)
Cost	<u>Open-source</u>	€0 direct licensing/paid-service expenditure vs uncharacterised post-trial Azure cost (Section 5.5)
SME Suitability (synthesis)	<u>Open-source</u>	Cost certainty and transparency outweigh CodeQL's depth advantage (Section 5.6)

Table 6.1: Summary of comparative results by metric

This study aimed to determine if a lightweight AI/ML-based pre-pipeline security gate could be integrated into a DevSecOps CI/CD pipeline with measurable gains in security outcomes for SMEs with minimal complexity and operations overhead (Section 1.2). To address this, two multi-layer security toolchains have been developed and analyzed using a common test artefact, a realistic, open-source toolchain (Semgrep, Trivy, Trufflehog, Falco, OPA) and an Azure cloud-native toolchain (CodeQL, Dependabot, Defender for Cloud, Sentinel, Azure Policy) with six metrics.

This study does not give an easy answer, of yes or no, and it is not a fact to be represented as such. If the question this study could test directly and directly, whether cloud-native tooling's structural, signature-based detection methods were superior to its open-source counterparts, the answer is yes: under the configurations in this study, the Coverage of Structural findings in the collected code, as provided by CodeQL, was significantly wider than that of the open-source tool, Semgrep (Section 5.1). But the key benefit did come at a tangible price: 9.7 times the baseline pipeline overhead (compared to the open-source stack's 3.5 times) and six documented setup friction points, vs. two for the open-source stack, and materially higher ongoing interpretive complexity (5 independent instances of cross-portal inconsistency across Sentinel, Azure Policy, and Defender for Cloud, Section 5.3, Section 5.6). In the absence of a dedicated

security engineer for these costs, this study's evidence backs the position that these costs offset CodeQL's detection-depth benefit, not to mention that the same structural detection was readily available in the open-source stack without the same cost and complexity due to tooling built by the open-source community (Semgrep, detection depth lower, no licensing costs, and less setup friction).

This study cannot definitively answer the question of whether AI/ML-enhanced behavioural and anomaly detection, specifically, has a proportionate benefit for deployment at the SME scale, as this was one of the claims it sought to advance in Chapter 1, and it does not make such a statement when it is not warranted. This difference, and its significance, are discussed directly in Section 6.2.

6.2 Strengths and Limitations

Here, with this study's adherence to the principles of Design Science Research, five limitations are explicitly stated:

First, the test artefact used in this study is a realistic, deliberately insecure learning application, as opposed to a purpose-built artefact with a foundation and a ground-truth labelled one like OWASP Benchmark. This positioning was intentional as Charoenwet et al. (2024) have shown that SAST tools work meaningfully worse when applied to real vulnerability-contributing code versus synthetic, fully labelled test programs, and this study's use of WebGoat is closer to real-world usage than a synthetic test would. But this also means that the detection capability results reported in this study (Section 5.1) cannot be reported against a full knowledge of the ground truth as a labelled benchmark would, and that the reported detection capability should therefore be read as what was observed with this particular test artefact, rather than as an exhaustive or statistically generalisable measure of the effectiveness of the tool. This study is based on the same generalisability limitation as the one Lipp, Banescu and Pretschner (2022) observed in their similar SAST study: the study is written in a single language (Java) and is based on a single test application (Spring Boot), thereby limiting the extent to which detection-capability and false positive results can be extended to SMEs with different technology stacks; it is treated here as a trade-off for in-depth, single-artefact evaluation design, rather than a mistake.

The second point is that this cost comparison (Section 5.5) was done during the 30-31 day free trial period of Azure Defender for Cloud and Azure Microsoft Sentinel. Both exhibited no actual costs in this study, but realistically continuing usage by SMEs in the post-trial setting was not observed. However, the study can only partially characterize the true cost comparison,

so that the €0-versus-€0 comparison reported in Chapter 5 should not be interpreted as a statement of equivalence in the long term between the two stacks.

Third, and most importantly for this study's initial motivation, the specific AI/ML-augmented functionality that this study's research question was centred on – behavioural correlation and anomaly detection – was never used in an adversarial scenario. This study was motivated in a direct way by the SolarWinds SUNBURST breach, an attack that evaded behaviour and anomaly detection, but this study was not able to build conditions to test Sentinel's behaviour detection capability in the same fashion, resulting in the class of threat that motivated this study's research question not being the same as the class of threat that the evidence this study produced is directly applicable to. During this study no telemetry source capable of producing attack-like behavioural signals was attached to Sentinel, and no data to correlate with was available for its Fusion and User and Entity Behaviour Analytics features. This is an important distinction this study carefully states with precision: the claim of the possibility of using AI/ML to detect the behaviour, the one this study goes to great pains to test, was not disproven, it was not tested because it was not possible to implement this claim in this study's scope and time. Taking this study's findings as evidence that AI/ML-augmented security tooling is not helpful when compared with structural analysis would overstate what was measured; its most unique stated benefit is empirically untested by this study when compared with structural analysis, and its distributive security capability is actually measured as a real cost of the Azure stack's structural detection tooling.

Fourth, this study's quantitative results — the false positive evidence base of six ground-truth data points (Section 5.2), and the three-trial pipeline-overhead comparison (Section 5.4) — are descriptive, not inferential. The sample size is too small for making confidence intervals, significance testing, or generalisations to the level of the population of DevSecOps tool comparisons, and the language used elsewhere in this study (for instance, H1 is confirmed) is not intended as a statistically validated generalisation to the level of the population of DevSecOps tool comparisons, but rather as consistent with the evidence observed within this study's specific scope. This focus was intentional as it gave this Master's dissertation the opportunity to study and examine the diagnostic investigation of each finding in greater detail rather than the repetition of trials a full inferential study would need to look at.

Fifth, the researcher has consistently implemented and evaluated all the work for this study without any independent second party verifying any findings. This reflects the individually-

authored nature of a Master's dissertation, but does not mean that there was no cross checking with a second, perhaps even differently-biased, observer; a finding common to many empirical software engineering studies in the past, but it is worth explicitly stating here rather than assuming it.

6.3 Relation to the Literature

The results of this study directly relate to some of the gaps found in Chapter 2. While they don't measure it directly, Rajapakse et al. (2022) cite tool-related friction as the top six-most reported barrier to DevSecOps adoption, which is an empirical measure of friction. In Section 5.3, it is reported that the top barrier to DevSecOps adoption is friction, in this case caused by tools, and this is a direct measure (not just a citation) of the barrier to DevSecOps adoption. The test programs used for evaluations of SAST tools in this study were not fully labelled to simplify the process of analysis, but rather realistic test programs from OWASP WebGoat. The use of realistic test programs was directly inspired by the argument of Charoenwet et al. (2024) and Lipp, Banescu and Pretschner (2022), which states that the use of synthetic test programs overstates the real-world performance of the tools, and the findings regarding the detection capabilities should be interpreted in that context. This is not a repetition of Wienczkowski (2026), who found a structural program-analysis techniques and adaptive, feedback-driven detection approaches disconnect at the level of a literature survey; this study's own result that Defender for Cloud significantly matches CodeQL's structural findings without adding any value to the detection process is a direct, quantified illustration of that separation, not a restatement. AI/ML-augmented security automation is simply a good enhancement of traditional tools, which is tested directly in this study, whereas Mahimalur et al. (2025) only claimed the advantage of AI/ML over traditional tools without conducting the empirical test itself.

6.4 Practical Implications for SMEs

The key implications for SMEs are summarized below. The key implications for SMEs are outlined below.

While this study offers evidence to inform an SME's choice between the two approaches, it is important to break down the six-metric comparison into practical adoption advice.

First, in selecting the tools, an SME should consider that these are the two dimensions that had the most obvious and the most consistently measured differences between the two stacks (as

explored in Sections 5.3, 5.4), and therefore should be taken into account when evaluating the complexity of setup and the added system overhead of using those tools. Secondly, the detection-depth advantage is indeed real, but should be considered along with the cost of using CodeQL in those pipelines: specifically, a team that is using CodeQL for its structural analysis depth should expect to see a corresponding increase in pipeline execution time, and the Maven build-step dependency that CodeQL introduces (Section 4.3.1) should be factored into the budget. Third, as a standalone feature, an SME is already using GitHub Actions and therefore they already have a centralised dashboarding solution, so the practical question is more about whether the dashboarding is worth the extra complexity than whether Azure is more secure than GitHub Actions (Section 4.3.3). Fourth, as discussed in Chapter 4 (Section 5.6), there are recurrent cross-portal inconsistencies, so any SME implementing the Azure stack should allow time for platform-specific troubleshooting, and the dashboard numbers should not be taken as gospel without more in-depth knowledge of the platforms. Last, and most directly to this study's original motivation, an SME specifically looking for behavioural or anomaly-detection capability against sophisticated, SUNBURST-style attacks should not assume the capability as taken-granted but should consider the cost of independent verification of the capability, or residual uncertainty identified in this study, when making decisions to adopt Sentinel.

6.5 Chapter Summary

In this chapter, they have all been discussed, related to the literature found in Chapter 2, and honestly discussed as limitations of this study as opposed to being left implicit. This study's contribution is presented in Chapter 7, directions for future work that are directly related to the limitations discussed here, and a closing statement.

7. Conclusions

While this study's findings, their relationship to the literature, and its strengths and limitations are discussed in Chapter 6, this chapter outlines this study's contribution, directions for future work, and a closing statement.

7.1 Contribution

The focus of this study is empirical, not theoretical. No other study could be found in the literature for this dissertation that builds, runs and compares an entire open source security toolchain to an entire Azure cloud-native security toolchain under controlled and measured conditions in front of the same representative test application, and not surveying practitioner opinion or evaluating tools individually. This directly tackles two gaps identified by Ccallo-Luque and Quispe-Quispe (2024): Firstly, Very Small Entities consistently struggle with validated and adapted guidance for adopting CI/CD tools, and, secondly, Rajapakse et al. (2022) found that tool-related challenges were the most common reported barrier to DevSecOps adoption through a systematic review but did not measure this challenge directly. Each of the six friction points found in the Azure stack (Section 5.3) has an identified root cause and a path to diagnosis that are real, verifiable examples of the type of adoption friction described in both papers but not in such detail.

A second contribution of the study is methodological: the ground-truth evidence base of paired true-positive and true-negative test cases per policy constructed in this study is an approach to false positive evaluation that extends and follows the 'type-by-type approach' outlined by Aloraini et al. (2019) as methodologically necessary and provides a replicable pattern for future tool evaluations in the context of SMEs. As well as being reported as resolved, the genuine implementation defects uncovered and overcome as part of this study are reported with sufficient diagnostic detail to be of use to a practitioner who observes the same platform behaviour—this is particularly true for the CI pipeline submodule bug that invalidated all automated pipeline results before 7 July 2026 (Section 4.4) and the Sentinel tenant-onboarding gap (Section 4.3.4).

The third contribution brings the two types of literature (prescriptive and measurement-oriented) that were identified in Chapter 2 together. This study deploys and tests these tools, which is not the case with many of the tools recommended by Dhandapani (2025) that were used for mapping CI/CD threats. Both Ayala and Garcia (2023) and Pan et al. (2024) agree in

the first place that adoption of CI/CD security tooling is inconsistent at population scale, and that attack surfaces can be found, but neither test the capabilities of any particular tool or toolchain; this study's six-metric comparison is a direct, executed response to the question these measurement-only studies leave open. Lipp, Banescu and Pretschner's (2022) methodological approach to real-world test artefacts and comparing tools to only a small number of categories is extended to the two-largest test category categories found in the security of application services, consisting of exactly 2 tool categories and exactly 10 tools, to form this full ten-component, two-stack comparison study. Altogether, this study is not simply another piece of the literature examined above, but provides the actual, actual evidence, that several of these papers explicitly say they lack.

7.2 Future Work

We do not make assertions about the generic extensions of the above limitations but rather provide the following five directions for future work.

First and foremost, from this study's most central untested claim, future research should experiment with a controlled, adversarial telemetry scenario — say simulated credential stuffing, unusual API call patterns, or a staged lateral-movement scenario — and introduce it to a connected Sentinel workspace, to see if the behavioural correlation engine identifies threats missed by structural, signature-based tooling. This would fill that particular gap identified by this study in Section 6.2 and would directly test the claim Mahimalur et al. (2025) make as an assumed benefit of the use of AI/ML augmented tooling, and not assume it.

Second, future studies should take this study a step further by either obtaining extended or paid Azure access to obtain a true post-trial cost comparison or modelling realistic SME-scale usage against published Azure pricing to provide a more genuine, multi-month cost comparison, rather than a snapshot of costs during the trial period alone.

Thirdly, this study's one-shot design comparing 2 different stacks to the same test artefact sacrificed breadth for depth and control. The design of this study may be extended to multiple applications of different language and architecture, with different friction points and trade-offs for the detection depth, to verify that the findings presented in this paper apply to other technology stacks that an SME could reasonably use.

Fourth, as noted in the scope and limitations of this study (Section 6.2), future research will require a significantly larger number of tests and policy classes to provide confidence intervals

and formal testing of whether a false positive or pipeline overhead is greater, which would then give full inferential evidence to support the current pilot-scale evidence base.

Finally, to determine if these results reported here can be reproduced by an independent researcher under similar circumstances, the build and compare methodology used in this study should be repeated in future research with a second researcher independently checking a sample of the data obtained; specifically, the data presented in this book for cross-portal inconsistencies in the data for more diagnostic inconsistencies, such as those found in Chapter 4, should be checked.

7.3 Closing Statement

The aim of this study was to determine whether the benefits of cloud-native AI/ML enhanced security tooling are commensurate with the cost for SMEs, not to assume it was. The evidence collected in six metrics and over two fully implemented and cross-validated toolchains provides a sound and defensible conclusion with respect to a specific claim this study was designed to test: For an SME without dedicated security staff, the complexity of the measured setup, the pipeline overhead, and the interpretive burden of the Azure cloud-native stack are real and substantial, and not compensated for by a demonstrated AI/ML-specific detection advantage, which was not exercised under the conditions this study could create. It is not a claim that cloud-native, AI/ML-augmented security tooling has no value in the context of SMEs; it is a claim that is directly supported by data from this study and is one that most heavily marketed in this space is still unproven until tested in a genuinely adversarial setting, the setting this study's future work in Section 7.2 seeks to provide.

REFERENCES

Aloraini, B., Nagappan, M., German, D.M., Hayashi, S. and Higo, Y. (2019) 'An Empirical Study of Security Warnings from Static Application Security Testing Tools', Journal of Systems and Software, 158, 110427. <https://doi.org/10.1016/j.jss.2019.110427>, alternative - https://github.com/Rakeshu99/devsecops-research/blob/main/docs/PDF%20An%20empirical%20study%20of%20security%20warnings%20from%20static%20application%20security%20testing%20tools_3585.pdf

Aqua Security (2026) 'Trivy Documentation'. Available at: <https://trivy.dev/> (Accessed: July 2026).

Ayala, J. and Garcia, J. (2023) 'An Empirical Study on Workflows and Security Policies in Popular GitHub Repositories', Proceedings of the 1st IEEE/ACM International Workshop on Software Vulnerability (SVM), pp. 6-9.

Ccallo-Luque, M. and Quispe-Quispe, A. (2024) 'Adoption and Adaptation of CI/CD Practices in Very Small Software Development Entities: A Systematic Literature Review', arXiv:2410.00623. <https://arxiv.org/abs/2410.00623>

Chandramouli, R., Kautz, F. and Torres-Arias, S. (2024) 'Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines', NIST Special Publication 800-204D. Available at: <https://doi.org/10.6028/NIST.SP.800-204D/>

Charoenwet, W., Thongtanunam, P., Pham, V.T. and Treude, C. (2024) 'An Empirical Study of Static Analysis Tools for Secure Code Review', Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 691-703. <https://arxiv.org/abs/2407.12241>

Cheenepalli, J., Hastings, J.D., Ahmed, K.M. and Fenner, C. (2025) 'Advancing DevSecOps in SMEs: Challenges and Best Practices for Secure CI/CD Pipelines', 2025 13th International Symposium on Digital Forensics and Security (ISDFS), pp. 1-6. <https://arxiv.org/abs/2503.22612>

Dhandapani, S. (2025) 'Enhancing Software Supply Chain Security through STRIDE-Based Threat Modelling of CI/CD Pipelines', arXiv:2506.06478. Available at: <https://arxiv.org/abs/2506.06478/>

European Union (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation). Official Journal of the European Union, L119, pp. 1-88. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/> (Accessed: August 2026).

FireEye (2020) 'Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor'. Available at: <https://www.fireeye.com/blog/threat-research/2020/12/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backdoor.html/>

(Accessed: August 2026).

GitHub (2026a) 'About code scanning with CodeQL'. Available at: <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning-with-codeql/> (Accessed: August 2026).

GitHub (2026b) 'About Dependabot'. Available at: <https://docs.github.com/en/code-security/dependabot/> (Accessed: August 2026).

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design Science in Information Systems Research', MIS Quarterly, 28(1), pp. 75-105. <https://doi.org/10.2307/25148625>

Lipp, S., Banescu, S. and Pretschner, A. (2022) 'An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection', Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 544-555. <https://doi.org/10.1145/3533767.3534380>

Mahimalur, R.K., Amgothu, S., Reddy, B.R.T. and Gadde, S.S. (2025) 'Modern Cloud Security and Automation: A DevSecOps Approach Leveraging AI/ML and Containerization', 2025 9th International Conference on Electronics, Communication and Aerospace Technology (ICECA), pp. 305-312.

Microsoft (2026a) 'Microsoft Defender for Cloud documentation'. Available at: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/> (Accessed: August 2026).

Microsoft (2026b) 'Microsoft Sentinel documentation'. Available at: <https://learn.microsoft.com/en-us/azure/sentinel/> (Accessed: August 2026).

Microsoft (2026c) 'Azure Policy documentation'. Available at: <https://learn.microsoft.com/en-us/azure/governance/policy/> (Accessed: August 2026).

Microsoft (2026d) 'Transition from grouped to individual recommendations in Defender for Cloud'. Available at: <https://learn.microsoft.com/en-us/azure/defender-for-cloud/transition-grouped-individual-recommendations/> (Accessed: August 2026).

Myrbakken, H. and Colomo-Palacios, R. (2017) 'DevSecOps: A Multivocal Literature Review', International Conference on Software Process Improvement and Capability Determination (SPICE), pp. 17-29. https://doi.org/10.1007/978-3-319-67383-7_2

Nair, A., Nicolazzo, S., Nocera, A., Rafidha Rehiman, K.A. and Vinod, P. (2025) 'SoK: Zero Trust as a Strategy to Address DevSecOps Challenges', 2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), pp. 546-554.
Open Policy Agent (2026) 'OPA Documentation'. Available at: <https://www.openpolicyagent.org/docs/latest/> (Accessed: August 2026).

OWASP (2022) Top 10 CI/CD Security Risks. Available at: <https://owasp.org/www-project-top-10-ci-cd-security-risks/> (Accessed: April 2026).

Pan, Z., Shen, W., Wang, X., Yang, Y., Chang, R., Liu, Y., Liu, C., Liu, Y. and Ren, K. (2024) 'Ambush from All Sides: Understanding Security Threats in Open-Source Software CI/CD Pipelines', IEEE Transactions on Dependable and Secure Computing, 21(1), pp. 403-418. <https://doi.org/10.1109/TDSC.2023.3253572>

Rajapakse, R.N., Zahedi, M., Babar, M.A. and Shen, H. (2022) 'Challenges and solutions when adopting DevSecOps: A systematic review', Information and Software Technology, 141, 106700. <https://arxiv.org/abs/2103.08266>

Semgrep (2026) 'Semgrep Documentation'. Available at: <https://semgrep.dev/docs/>

(Accessed: August 2026).

The Falco Authors (2026) 'Falco Documentation'. Available at: <https://falco.org/docs/>

(Accessed: August 2026).

Truffle Security (2026) 'TruffleHog'. Available at: <https://github.com/trufflesecurity/trufflehog/> (Accessed: August 2026).

Wienczkowski, M. (2026) 'Adaptive and AI-Augmented Security Testing: A Systematic Survey of Program Analysis, Feedback-Driven Testing, and Hybrid Learning-Based Approaches', arXiv:2604.27000. Available at: <https://arxiv.org/abs/2604.27000/>

(Accessed: August 2026).

APPENDICES

Appendices are named alphabetically and paginated consecutively, separately from the main text, consistent with the style guide. All source files reproduced here are the author's own, and are also available in full within the linked repository (Section 3.2).

Appendix A: OPA Rego Policies

The three Rego policies underlying the OPA false-positive-rate evidence base reported in Table 5.2 are reproduced below in full, each corresponding to a distinct OWASP CI/CD Security Top 10 risk class as discussed in Section 4.2.5.

A.1 — CICD-SEC-1: Insufficient Flow Control (github-actions-security.rego)

```
package githubactions

import rego.v1

# Deny rule: flags any 'run:' step that directly interpolates
# untrusted github context data without using an intermediate
# environment variable - the same risk class Semgrep flagged
# in WebGoat's release.yml (yaml.github-actions.security.run-shell-injection)

deny contains msg if {
    some job_name, job in input.jobs
    some step in job.steps
    step.run
    contains(step.run, "{{ github.}")
    msg := sprintf("Job '%s' has a 'run' step using untrusted github context data directly: %s", [job_name, step.run])
}
```

Listing A.1: github-actions-security.rego

A.2 — CICD-SEC-3: Dependency Chain Abuse (action-pinning.rego)

```
package actionpinning

import rego.v1

deny contains msg if {
    some job_name, job in input.jobs
    some step in job.steps
    step.uses
    not regex.match(`@[0-9a-f]{40}$`, step.uses)
    not startswith(step.uses, "actions/")
    not startswith(step.uses, ".")
    msg := sprintf("Job '%s' uses unpinned third-party action: %s", [job_name, step.uses])
}
```

Listing A.2: action-pinning.rego

A.3 — CICD-SEC-6: Insufficient Credential Hygiene (secret-logging.rego)

```
package secretlogging

import rego.v1

deny contains msg if {
    some job_name, job in input.jobs
    some step in job.steps
    step.run
    contains(step.run, "echo")
    contains(step.run, "{{ secrets.}")
}
```

```

    msg := sprintf("Job '%s' echoes a secret value directly to logs: %s", [job_name, step.run])
}

```

Listing A.3: secret-logging.rego

Appendix B: Baseline CI/CD Pipeline Workflow

The baseline pipeline (Section 5.4), used as the control condition for measuring pipeline overhead against both toolchains, is reproduced below in full. The full open-source and Azure stack pipeline definitions are longer and are available in the linked repository rather than reproduced here in full, consistent with keeping appendix material representative rather than exhaustive.

```

name: Baseline Pipeline — No Security Tooling

on:
  push:
    branches: [ main ]
    paths:
      - 'app/**'
      - '.github/workflows/baseline.yml'
  pull_request:
    branches: [ main ]
    paths:
      - 'app/**'
      - '.github/workflows/baseline.yml'
  workflow_dispatch:

jobs:
  baseline-build:
    name: Baseline Build (Control Condition)
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Record pipeline start time
        run: echo "Pipeline started at $(date -u +%Y-%m-%dT%H:%M:%SZ)"

      - name: Simulate application build
        run: |
          echo "Simulating application build step"
          echo "No security tooling applied in this baseline pipeline"
          echo "This pipeline serves as the control condition for measuring pipeline overhead"
          sleep 5

      - name: Record pipeline end time
        run: echo "Pipeline completed at $(date -u +%Y-%m-%dT%H:%M:%SZ)"

```

Listing B.1: baseline.yml — control-condition GitHub Actions workflow